

RayTracing

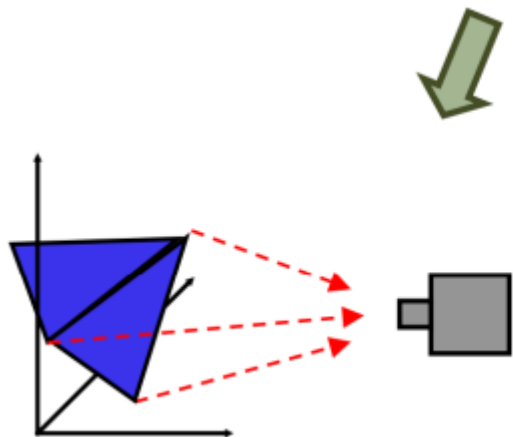
**GPU Ray Tracing Vladimir Frolov
(Nvidia, MSU, Keldysh Institute of
Applied Math)**

План

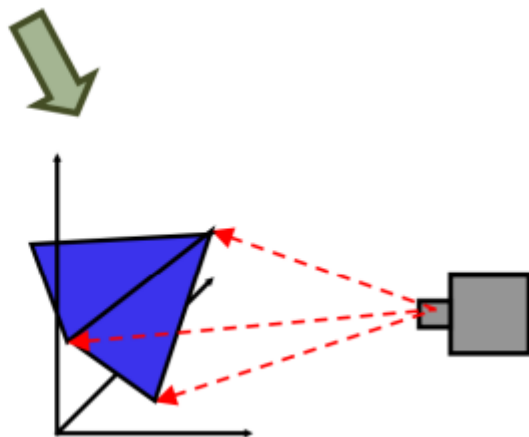
- Трассировка лучей и растеризация
 - Определение видимости
 - Global illumination
- Ускоряющие структуры
- Особенности трассировки лучей на GPU
- Узкие места
- Работа мультипроцессора на низком уровне
 - CPU vs GPU
- Оптимизация и отладка неоднородного кода

Растеризация vs трассировка

Два основных подхода



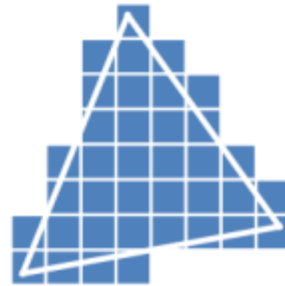
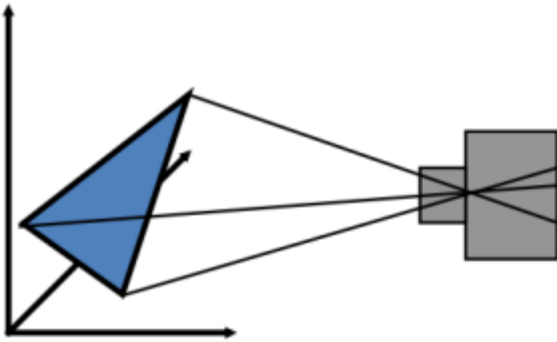
Растеризация:
прямая проекция геометрии



Трассировка лучей:
обратная проекция
пикселей изображения

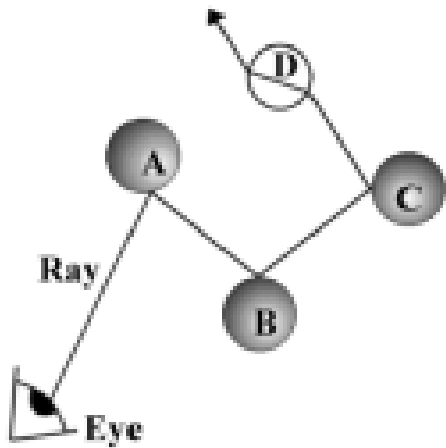
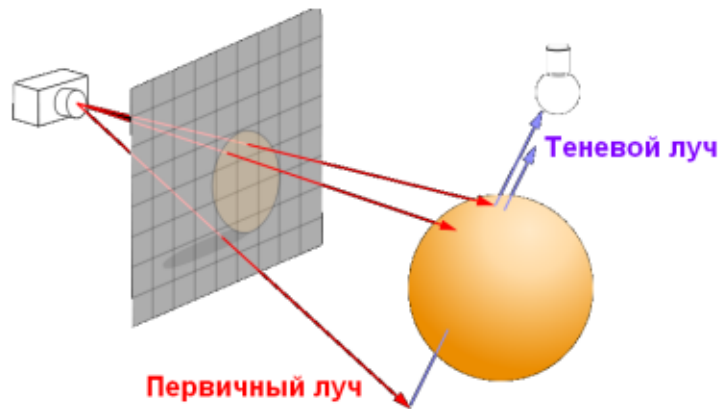
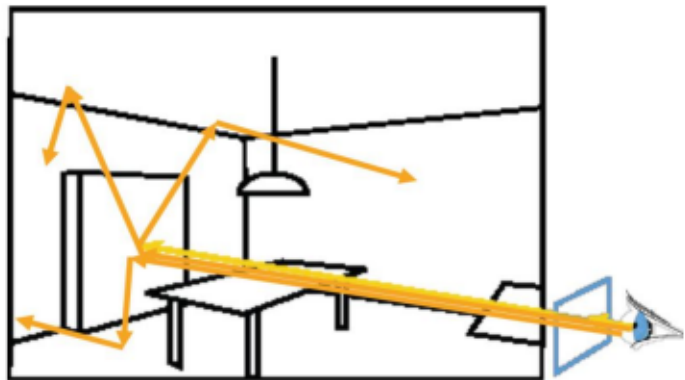
Растеризация vs трассировка

- Растеризация
 - Обработка по 1 треугольнику
 - Трудно параллелится
 - Эквивалентна задаче построения дерева или сортировке



Трассировка лучей

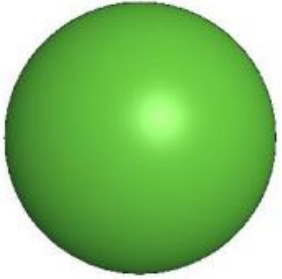
- Алгоритм



Трассировка лучей

Представление 3D объектов:

- Аналитическое



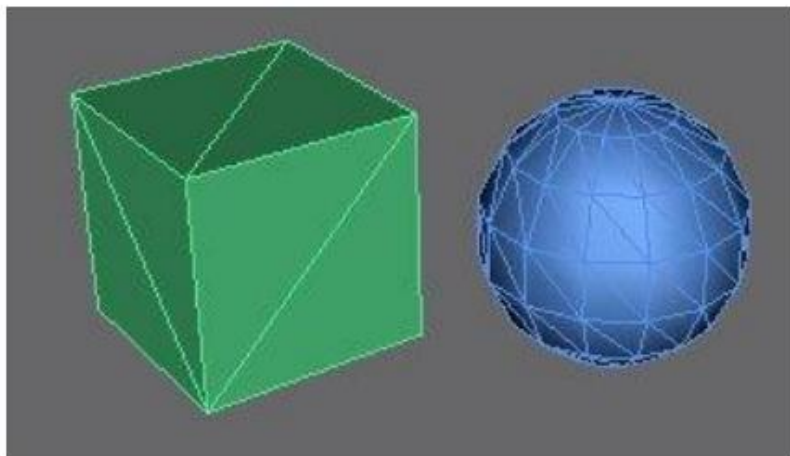
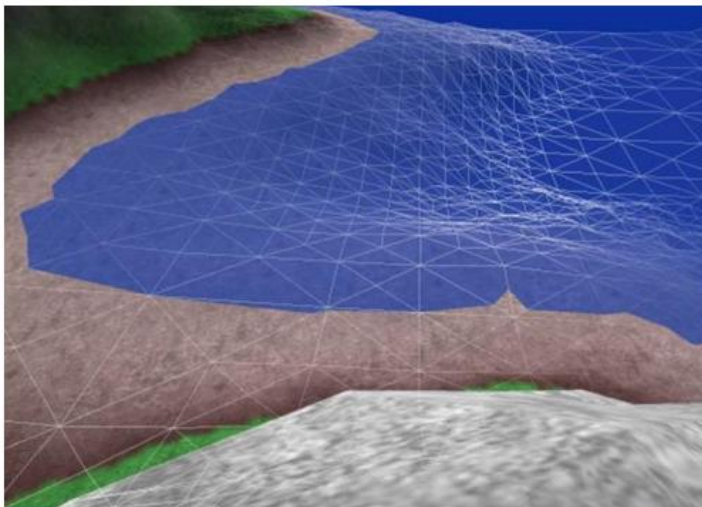
- Меши из треугольников



Трассировка лучей

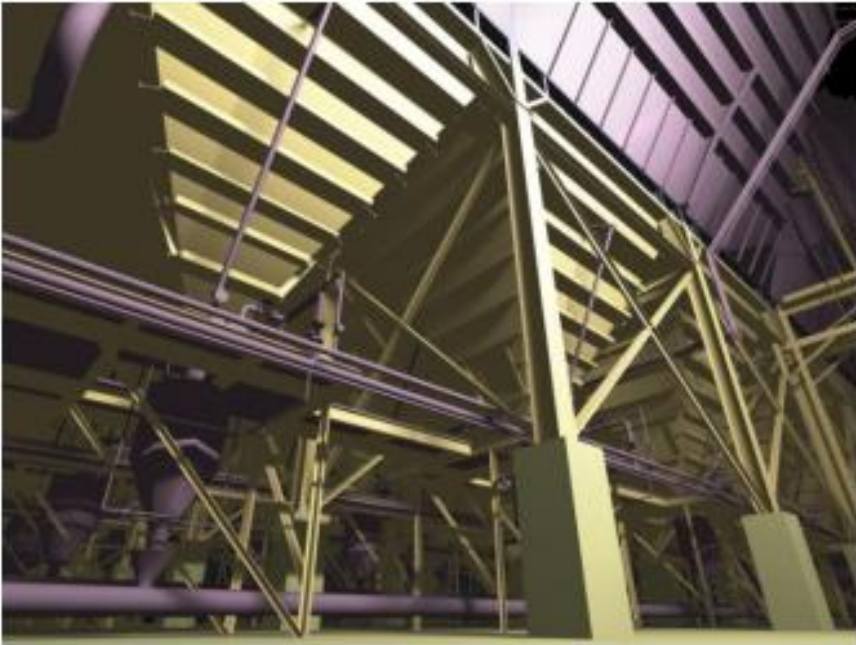
Представление 3D объектов:

- Меши из треугольников



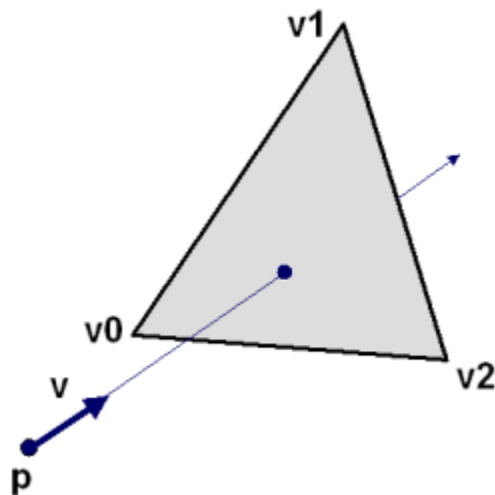
Трассировка лучей

- Поверхность задана как массив треугольников
- Узкое место – поиск пересечения луча с поверхностью
 - 1 000 000 треугольников
 - 1 000 000 лучей
 - $\Rightarrow 10^{12}$ операций
 - $\log(N)^k * 10^6$ ($k \sim [1..2]$)



Пересечение луча и треугольника

- Простой вариант
 - $Ax + By + Cz + D = 0$
 - Найти t
 - $x = p.x + v.x * t$
 - $y = p.y + v.y * t$
 - $z = p.z + v.z * t$

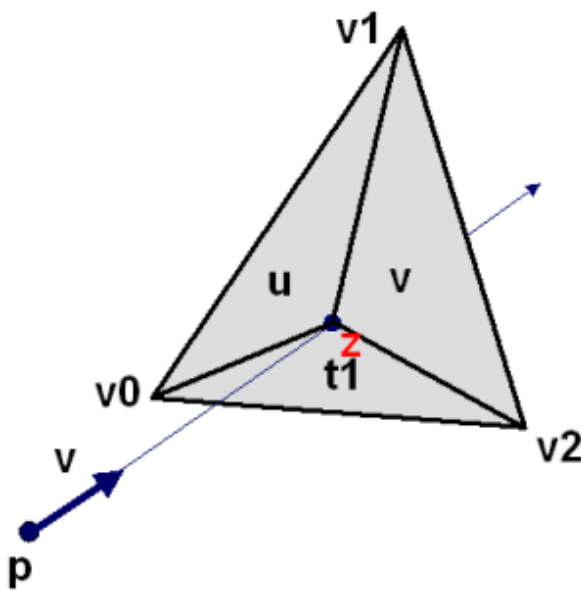


$$t = - \frac{(A * p.x + B * p.y + C * p.z + D)}{A * v.x + B * v.y + C * v.z}$$

Пересечение луча и треугольника

- Простой вариант
 - t известно
 - $z = p + v * t$
 - $S = \text{cross}(v1 - v0, v2 - v0)$
 - $u = \text{cross}(v1 - z, v0 - z)$
 - $v = \text{cross}(v1 - z, v2 - z)$
 - $t1 = \text{cross}(v2 - z, v0 - z)$

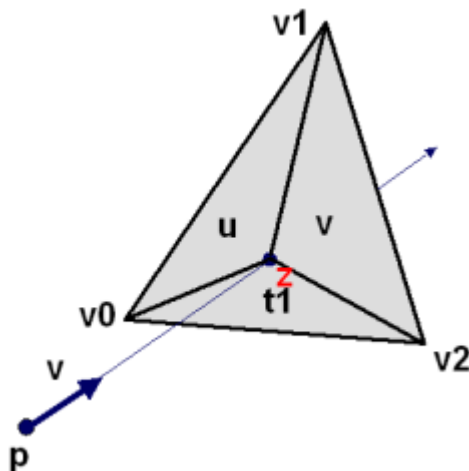
$$- |u + v + t1 - S| < \varepsilon$$



Пересечение луча и треугольника

- Оптимизированный вариант
 - Барицентрические координаты:

- $u := u / S$
- $v := v / S$
- $t1 := t1 / S$
- $t1 = 1 - u - v$



$$z(u, v) = (1 - u - v) * v1 + u * v2 + v * v0$$

$$z(t) = p + t * d$$

$$p + t * d = (1 - u - v) * v1 + u * v2 + v * v0$$

Пересечение луча и треугольника

- Оптимизированный вариант

$$\begin{bmatrix} t \\ u \\ v \end{bmatrix} = \frac{1}{\text{dot}(P, E1)} * \begin{bmatrix} \text{dot}(Q, E2) \\ \text{dot}(P, T) \\ \text{dot}(Q, D) \end{bmatrix}$$

$$E1 = v1 - v0$$

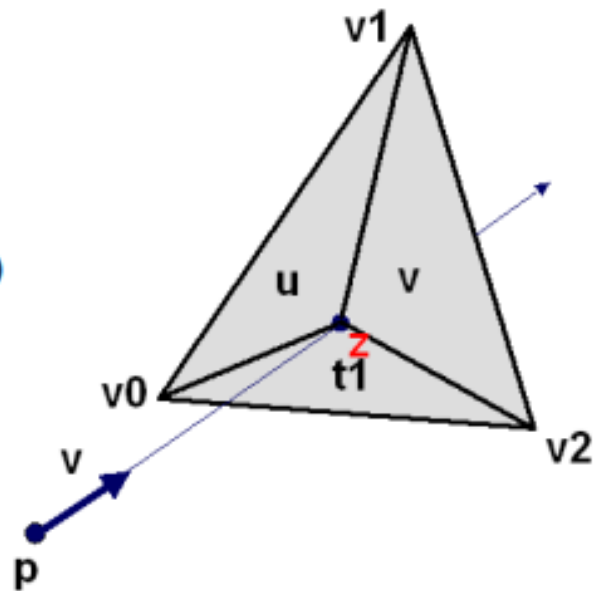
$$E2 = v2 - v0$$

$$T = p - v0$$

$$P = \text{cross}(D, E2)$$

$$Q = \text{cross}(T, E1)$$

$$D = v$$



Пересечение луча и треугольника

- Unit test

float3 o = mul3x4(m, origin);

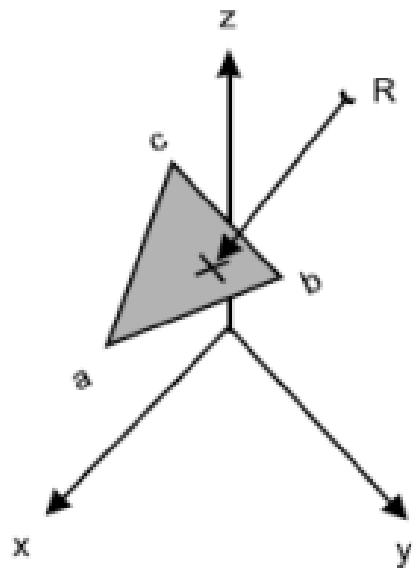
float3 d = mul3x3(m, dir);

float t = -o.z / d.z;

float u = o.x + t * d.x;

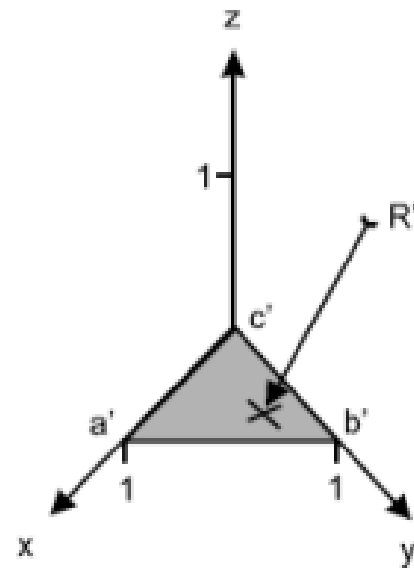
float v = o.y + t * d.y;

Где m это T_{Δ}



(a) world coordinate space

affine triangle
transformation



(b) unit triangle space

Пересечение луча и треугольника

- Unit test

float3 o = mul3x4(m, origin);

float3 d = mul3x3(m, dir);

float t = -o.z / d.z;

float u = o.x + t * d.x;

float v = o.y + t * d.y;

Где m это T_{Δ}

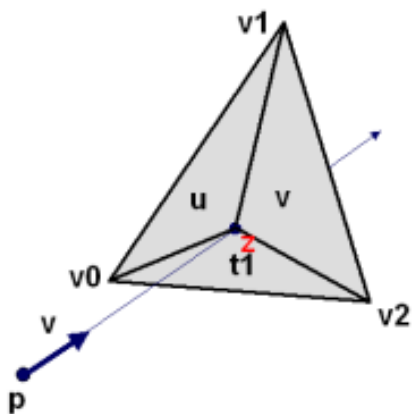
$$T_{\Delta}^{-1}(X) = \begin{pmatrix} A_x - C_x & B_x - C_x & N_x - C_x \\ A_y - C_y & B_y - C_y & N_y - C_y \\ A_z - C_z & B_z - C_z & N_z - C_z \end{pmatrix} \cdot X + \begin{pmatrix} C_x \\ C_y \\ C_z \end{pmatrix}$$

$$T_{\Delta}^{-1} \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} = A \quad T_{\Delta}^{-1} \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} = B$$

$$T_{\Delta}^{-1} \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = C \quad T_{\Delta}^{-1} \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = N$$

Пересечение луча и треугольника

- Простой вариант
 - Операции ($*$: 39, $+/-$: 53, $/$: 1)
- Оптимизированный вариант
 - Операции ($*$: 23, $+/-$: 24, $/$: 1)
- Юнит тест
 - Операции ($*$: 20, $+$: 20, $/$: 1)
 - Экономит регистры GPU



Занятость мультипроцессора

- Регистры
 - GF100: maxreg = 32768, maxwarps = 48
 - block size = 256
 - рег <= 40: 3 блока, 24 warp-а активны
 - рег <= 32: 4 блока, 32 warp-а активны
 - рег <= 24: 5 блоков, 40 warp-ов активны
 - рег <= 20: 6 блоков, 48 warp-ов активны

$$\text{occupancy} = \frac{\text{active warps}}{\text{max warps}}$$

GPU vs CPU

- Локальные переменные
 - CPU : стэк
 - GPU : регистры
- Задержки в конвейере (Pipeline hazards)
 - CPU : Out Of Order
 - GPU : Hyperthreading
- Латентность памяти
 - CPU: огромный кэш
 - GPU: сокращение латентности за счет параллелизма

Проблемы конвейерных ВУ

add R1,R2
mul R0,R1



add R1,R2
nop
nop
mul R0,R1

mul R0,R1

add R1,R2

□

Fetch

Decode

Execute

Memory

Writeback



Регистровый файл

Read After Write (RAW) Hazard

Почему GPU быстрее?

add R1,R2
mul R0,R1

add R1,R2 *
mul R0,R1 *

add R1,R2 **
mul R0,R1 **

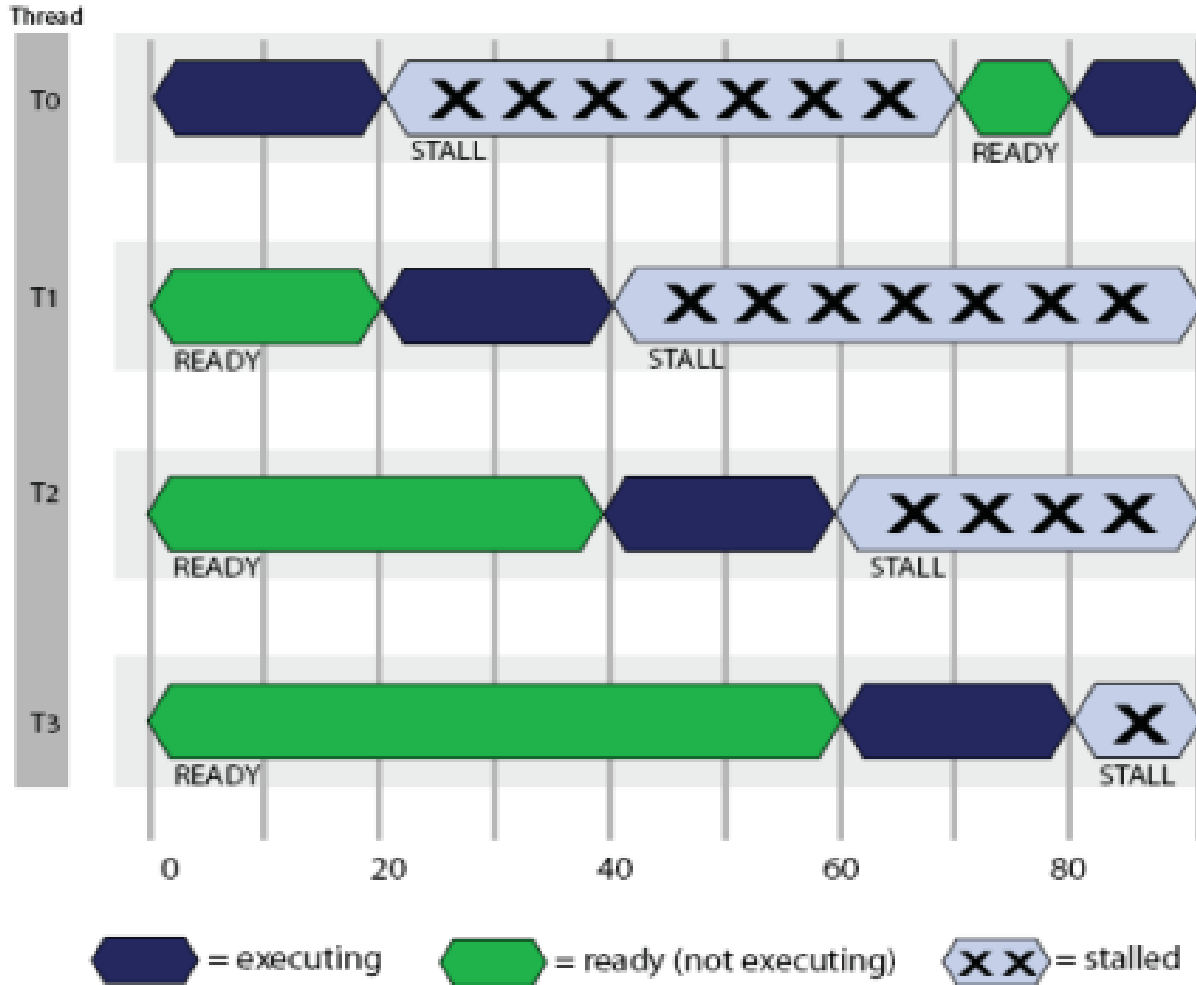


add R1,R2
add R1,R2 *
add R1,R2 **

mul R0,R1
mul R0,R1 *
mul R0,R1 **

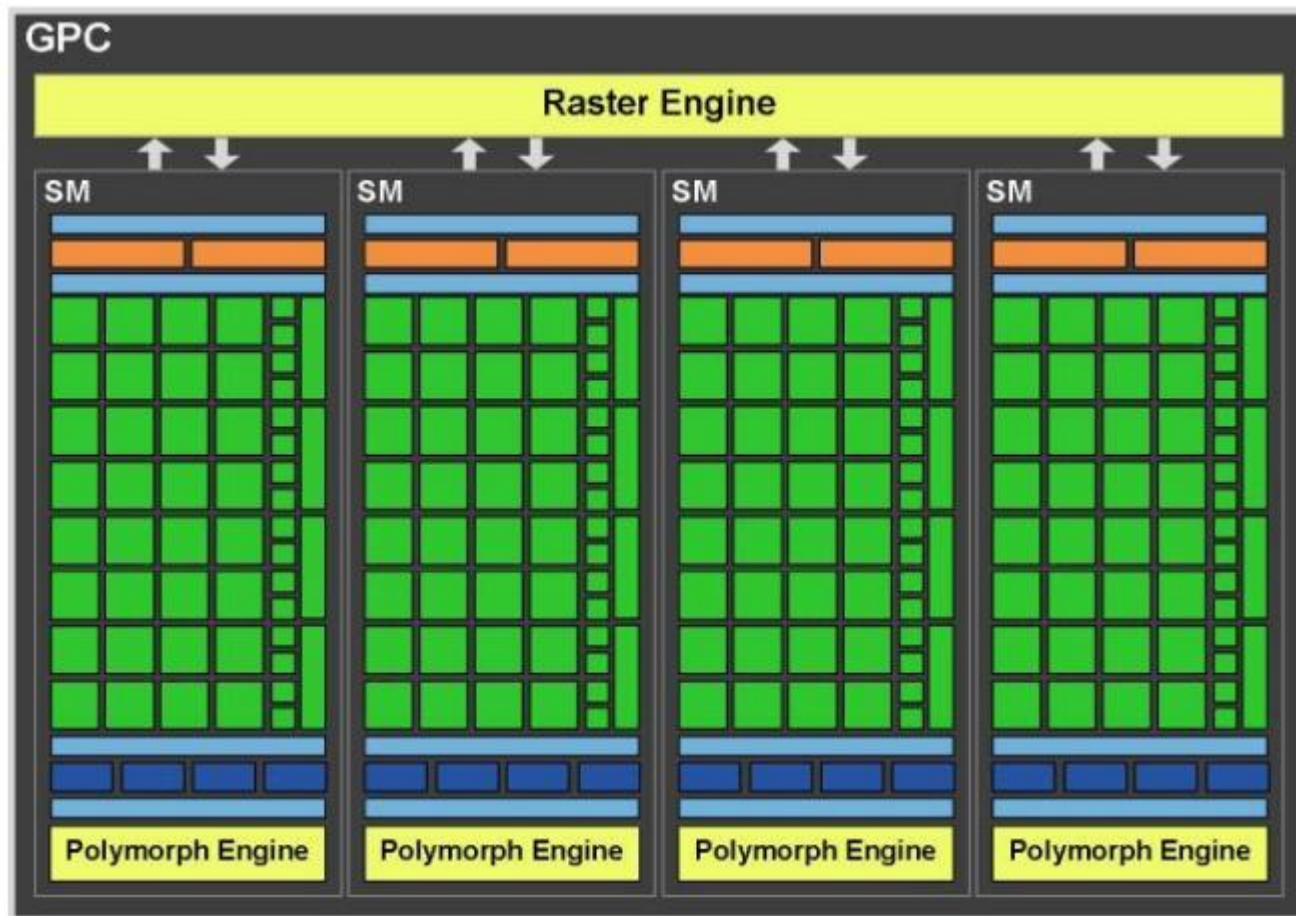
GPU vs CPU

- Латентность памяти



Занятость мультипроцессора

- Регистры



Занятость мультипроцессора

- Регистры
 - G80: maxreg = 8192, maxwarps = 24
 - GT200: maxreg = 16384, maxwarps = 32
 - GF1**: maxreg = 32768, maxwarps = 48

$$\text{occupancy} = \frac{\text{active warps}}{\text{max warps}}$$

Занятость мультипроцессора

- Регистры
 - GF100: maxreg = 32768, maxwarps = 48
 - block size = 256
 - рег <= 40: 3 блока, 24 warp-а активны
 - рег <= 32: 4 блока, 32 warp-а активны
 - рег <= 24: 5 блоков, 40 warp-ов активны
 - рег <= 20: 6 блоков, 48 warp-ов активны

$$\text{occupancy} = \frac{\text{active warps}}{\text{max warps}}$$

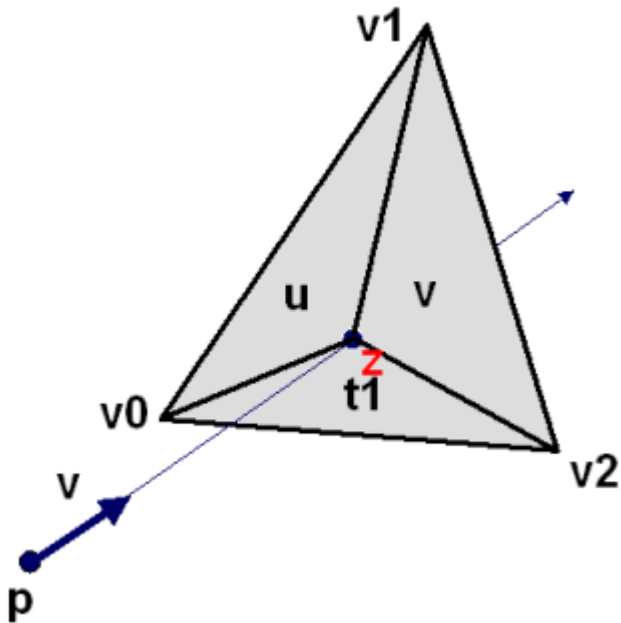
Занятость мультипроцессора

- Разделяемая память
 - GF100: size per SM = 49152 bytes, maxwarps = 48
 - block size = 256, 8 warps
 - 16384 байт на блок (64 байта на тред)
 - 12288 байт на блок (48 байт на тред)
 - 8192 байта на блок (32 байта на тред)

$$\text{occupancy} = \frac{\text{active warps}}{\text{max warps}}$$

Пересечение луча и треугольника

- Оптимизированный вариант
 - 24-32 регистра
- Unit test
 - 20-24 регистра
- И это только пересечение луча и треугольника!



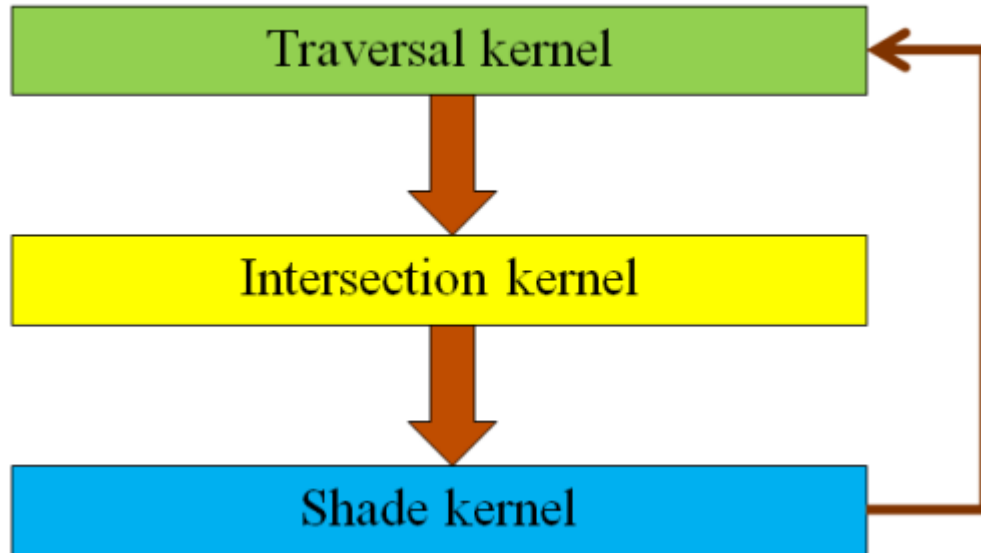
Экономим регистры

- Два подхода:
 - Uber kernel

```
__global__ void Uber_kernel(){  
    START:  
    if (state == TRAVERSE){  
        if (node.leaf()){  
            //save necessary data  
            state = INTERSECTION;  
            goto START;  
        }  
    }  
    else if (state == INTERSECTION){  
        // ...  
    }  
    else if (state == SHADE){  
        // ...  
    }  
}
```

Экономим регистры

- Два подхода:
 - Uber kernel
 - Separate kernel

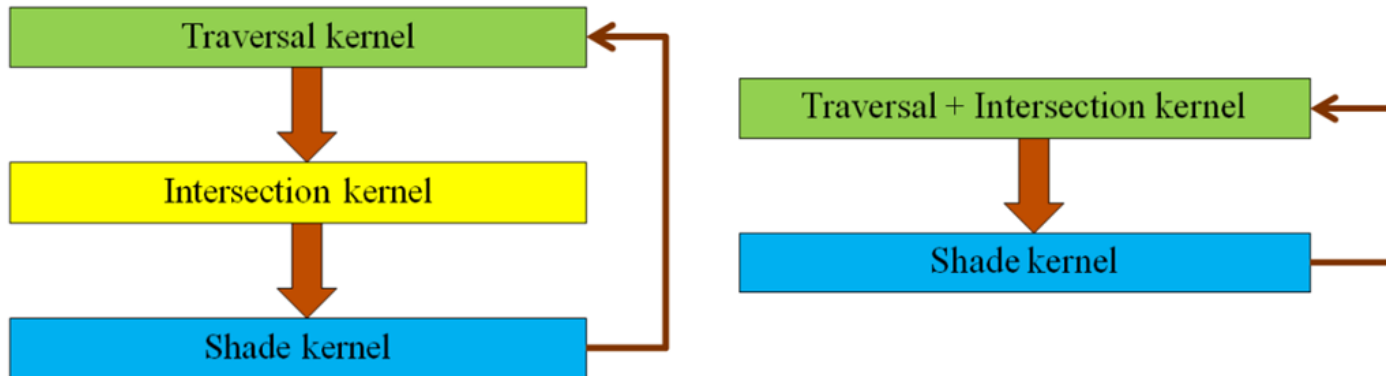


Uber kernel vs Separate kernel

- Uber kernel
 - Рекурсия (+)
 - Локальность данных (+)
 - Непрерывное выполнение (+)
 - Трудно профилировать (-)
 - Трудно оптимизировать (-)
 - Самая сложная часть лимитирует все ядро (-)
- Separate kernel
 - Нет рекурсии(-)
 - Обращения к DRAM (-)
 - Латентность остановки/запуска (-)
 - Легче отлаживать (+)
 - Легко профилировать (+)
 - Легко оптимизировать (+)
 - Гибкость (+)

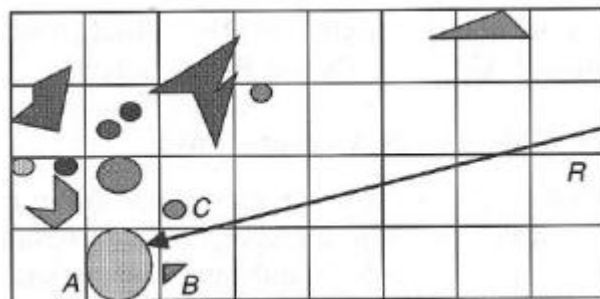
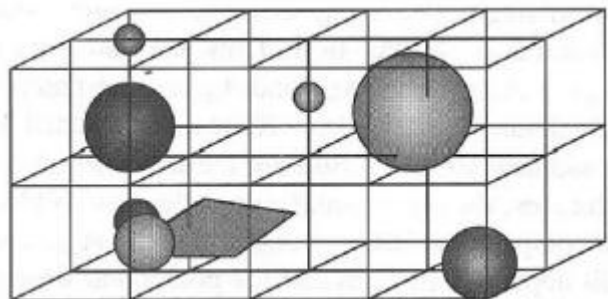
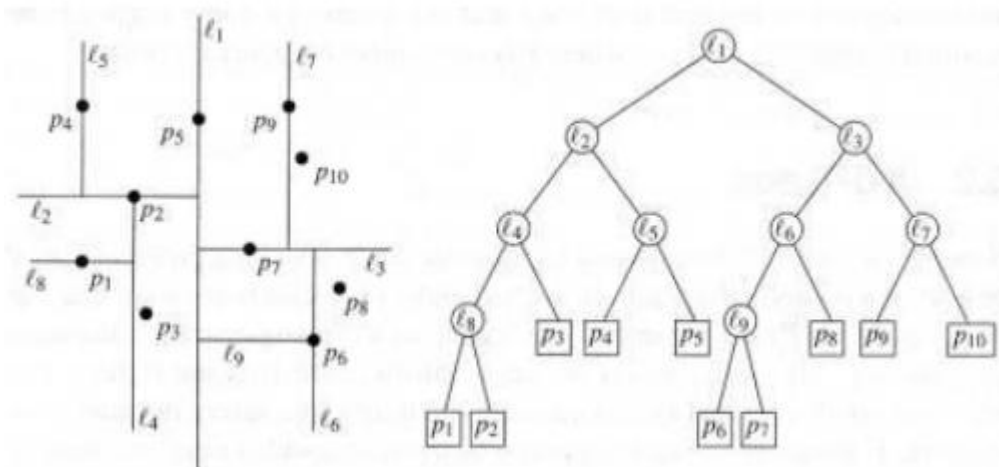
Организация рей трейсера

- Ядро пересечений – 24-32 регистра
- Но это только пересечения
- Нужно разбить алгоритм трассировки на несколько ядер



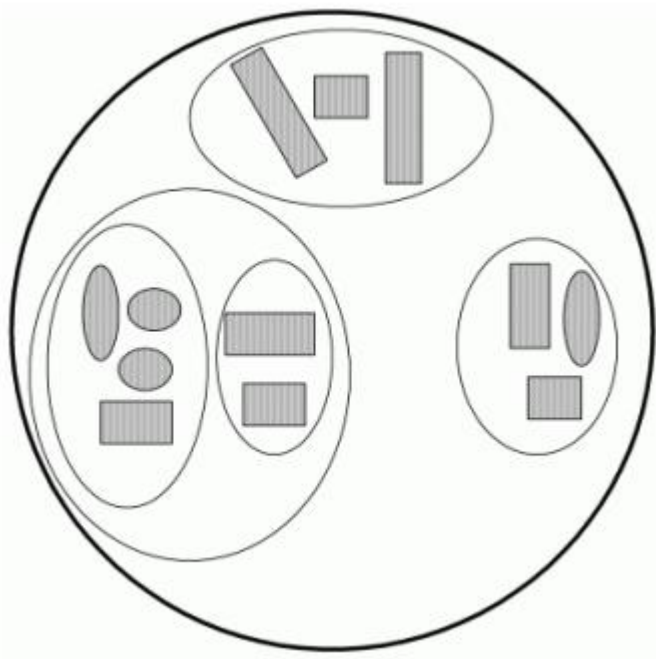
Ускоряющие структуры

- regular grid
- kd-tree
- BVH

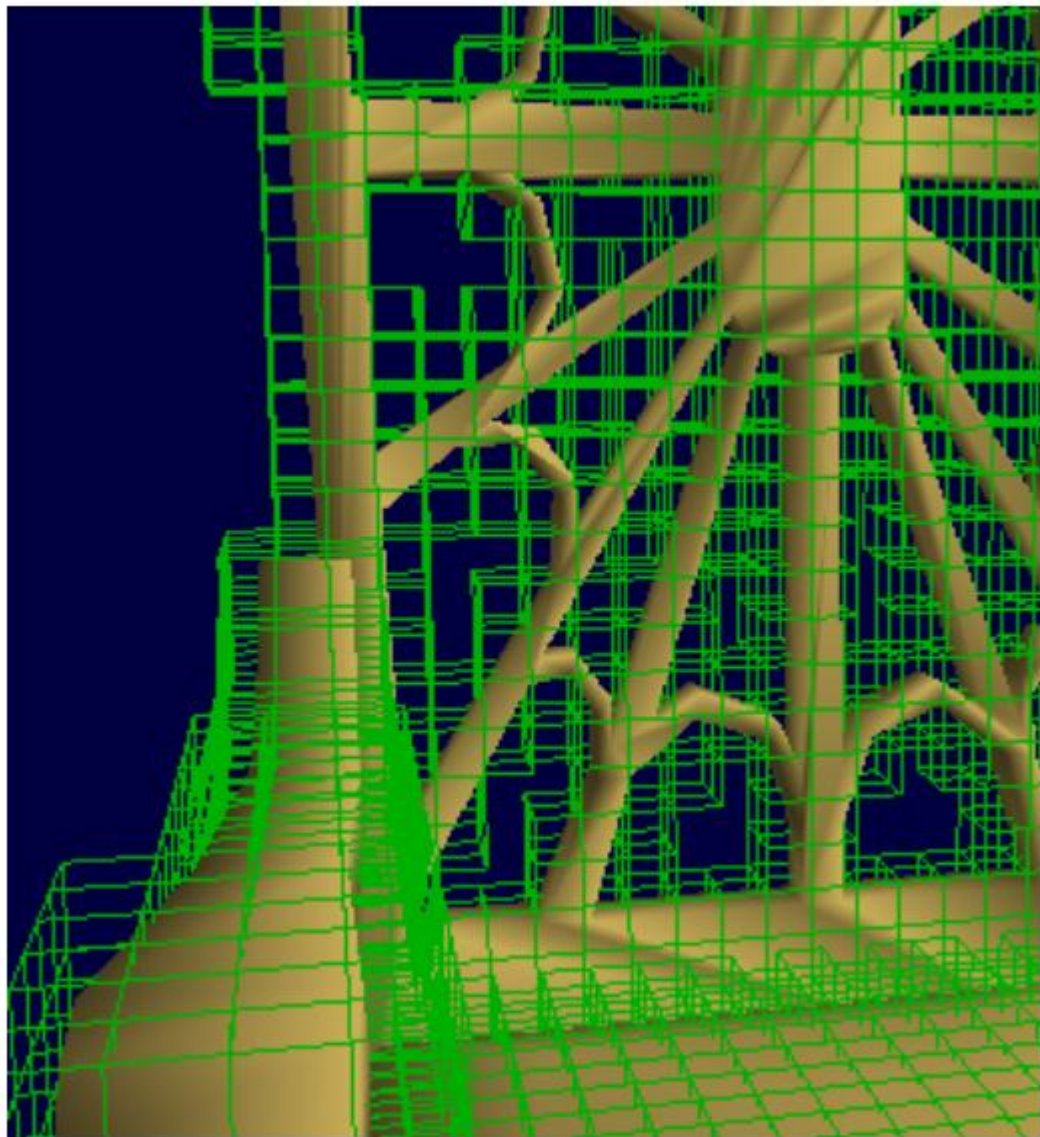


Ускоряющие структуры

- regular grid
- kd-tree
- BVH



Регулярная сетка



Регулярная сетка

- traversal

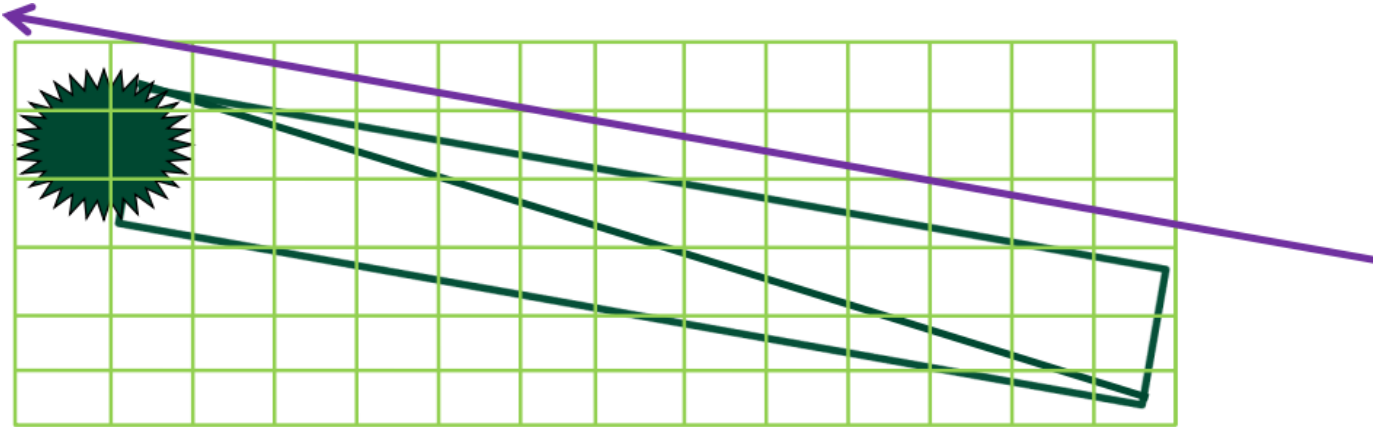
```
if (tMaxX <= tMaxY && tMaxX <= tMaxZ){  
    tMaxX += tDeltaX;  
    x += stepX;  
}  
else if (tMaxY <= tMaxZ && tMaxY <= tMaxX){  
    tMaxY += tDeltaY;  
    y += stepY;  
}  
else {  
    tMaxZ += tDeltaZ;  
    z += stepZ;  
}
```

Регулярная сетка

- Преимущества
 - Просто и быстро строится
 - Простой алгоритм траверса
- Недостатки
 - Плохо справляется с пустым пространством
 - Требуется много памяти
 - Много повторных пересечений – отвратительно разбивает геометрию
- Только для небольших сцен (1-50K)

Регулярная сетка

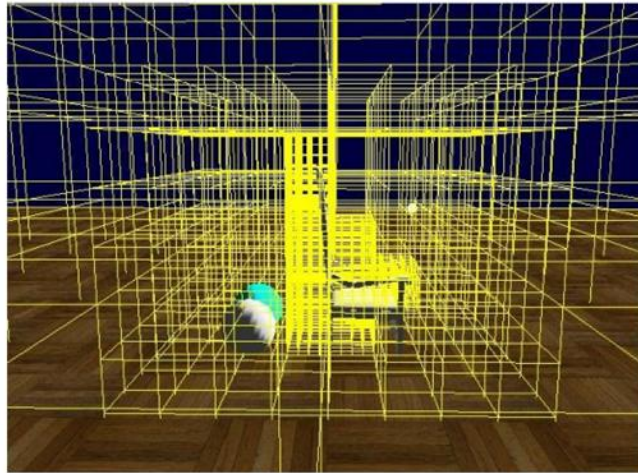
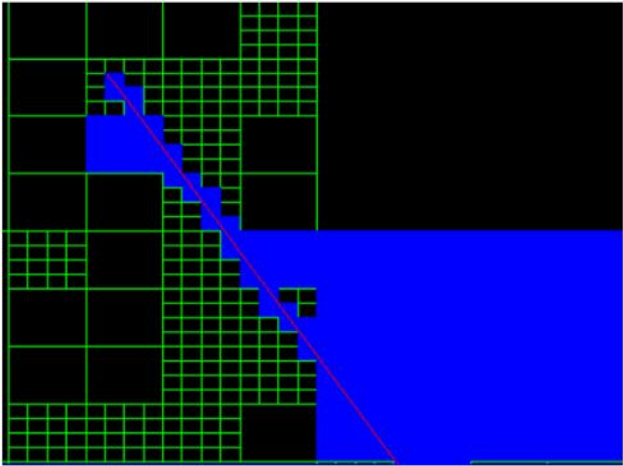
- Почему сетка плохо разбивает геометрию?



- Перебрали 15 вокселей
- 7 раз посчитали пересечение с одним и тем же треугольником!

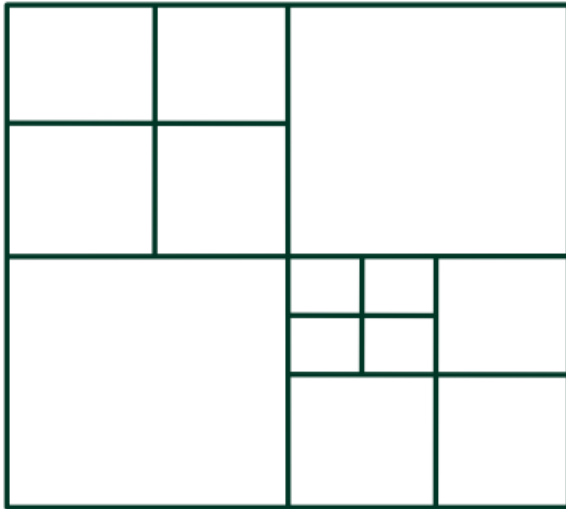
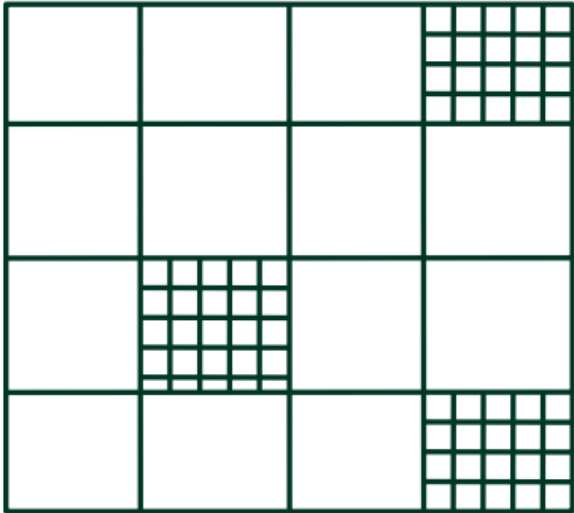
Иерархическая сетка

- Небольшое число вокселей
- Рекурсивно разбиваем воксели в местах с плотной геометрией

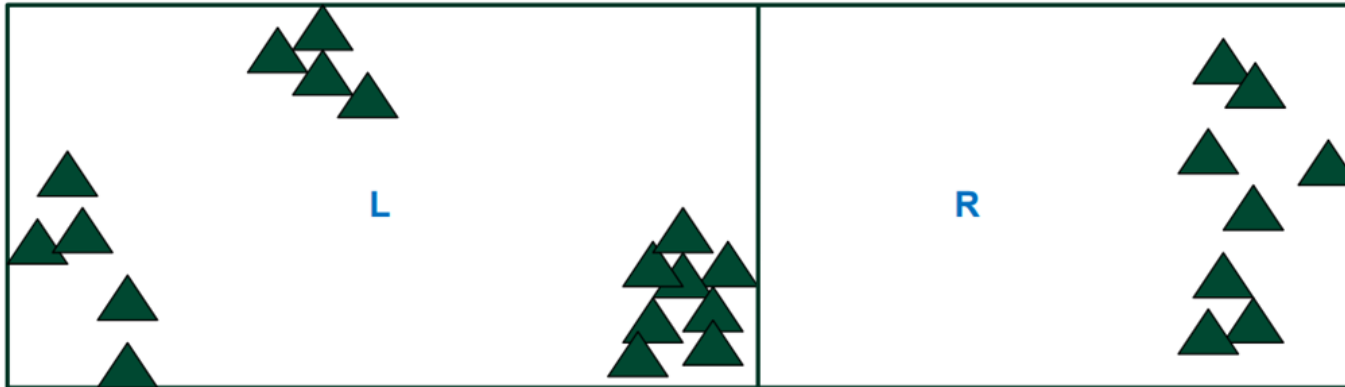
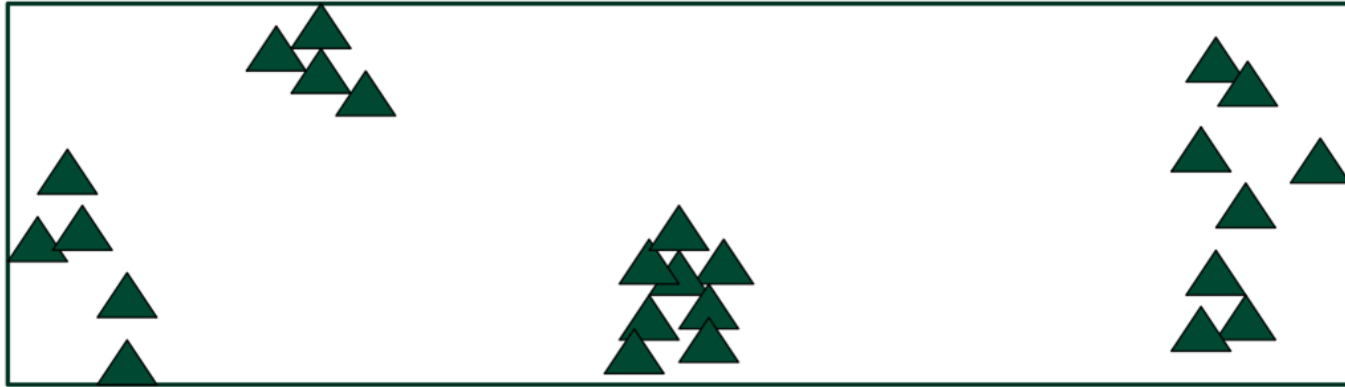


Иерархическая сетка

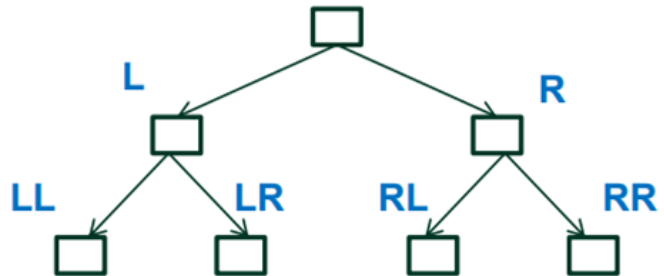
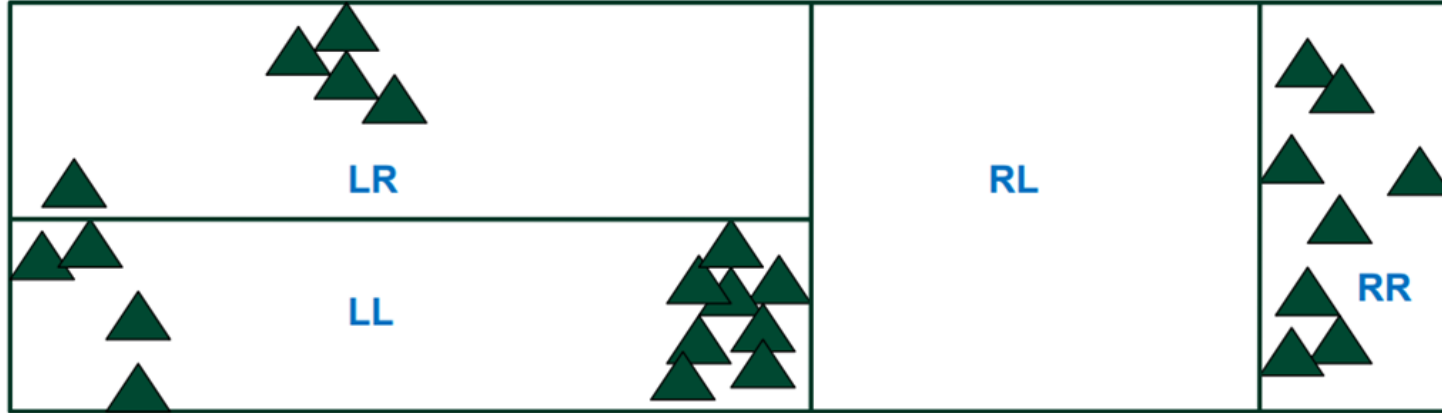
- Решает проблему чайника на стадионе (+)
- Переход между узлами вычислительно сложен (-)
- 12 доп. регистров как минимум (-)
- Нужно устранять рекурсию (-)



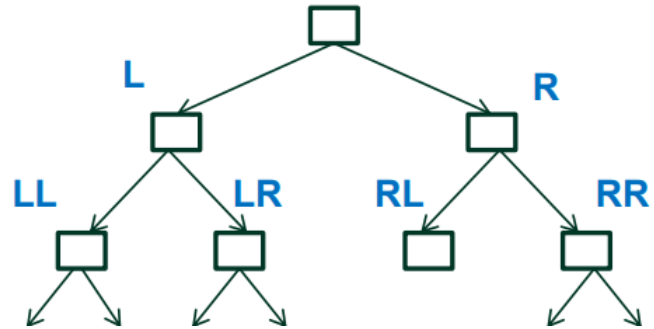
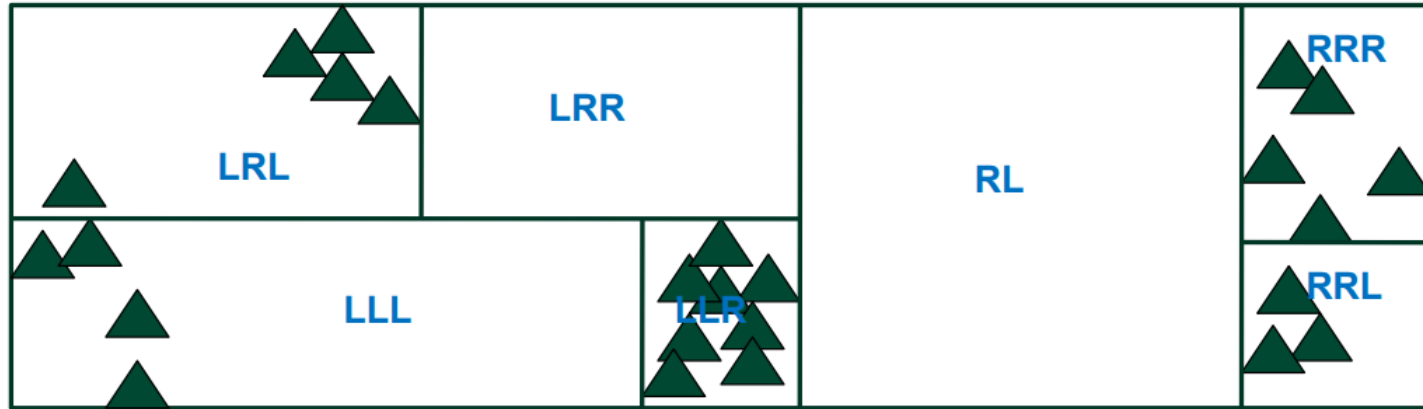
kd-деревья



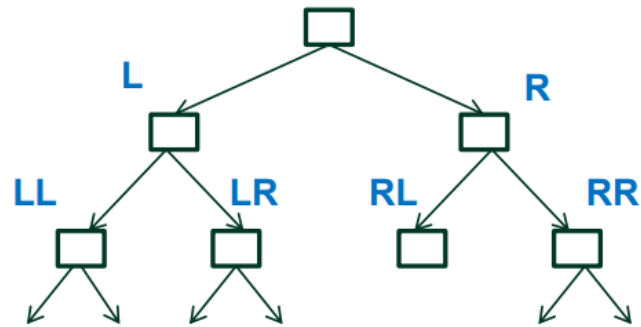
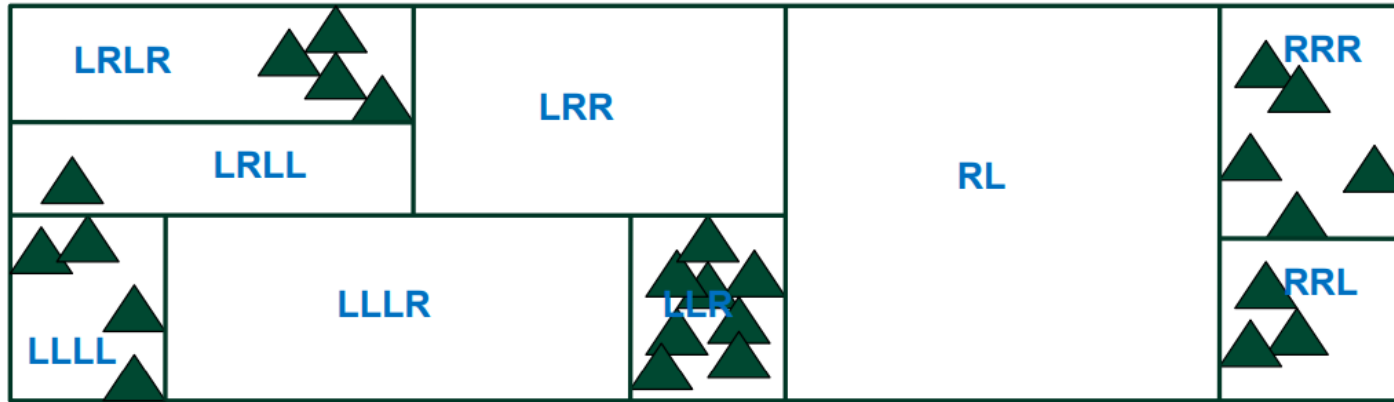
kd-деревья



kd-деревья

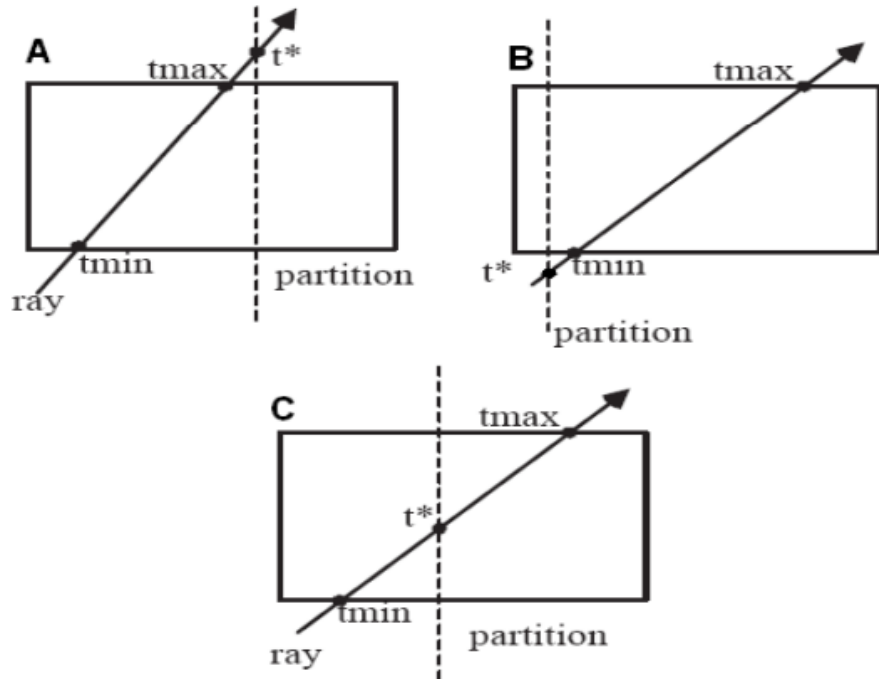


kd-деревья



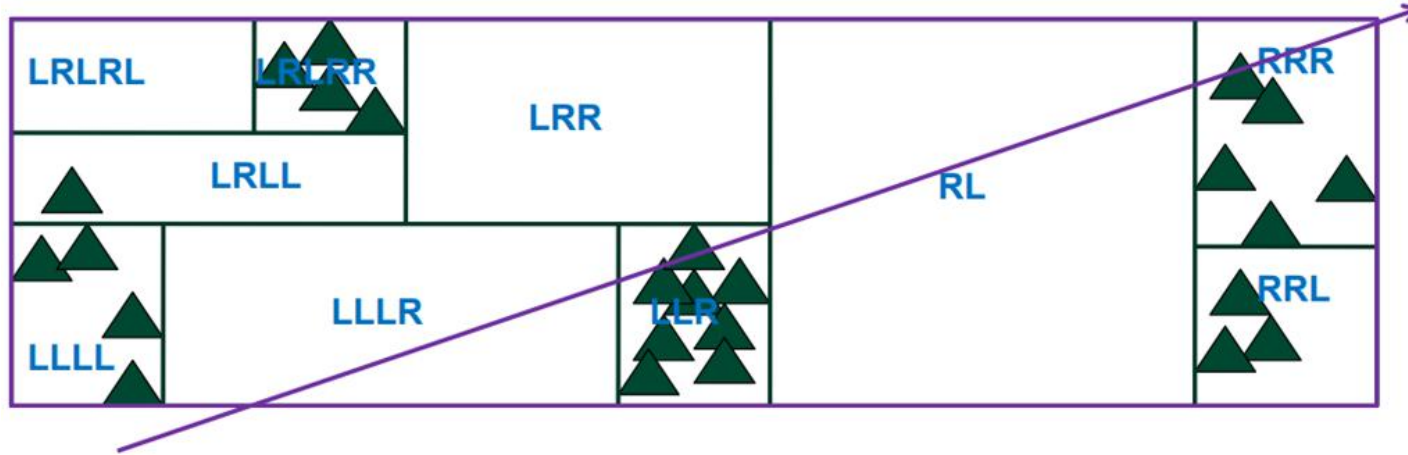
Иерархическая сетка

- Алгоритм траверса
- Регистры – 13 min:
 - луч - 6
 - t , $t_{\min}$, $t_{\max}$ – 3
 - node – 2
 - tid , $stack_top$ – 2
 - На практике удалось уложиться в 16!
 - Стек: локальная память



kd-деревья

- Алгоритм траверса

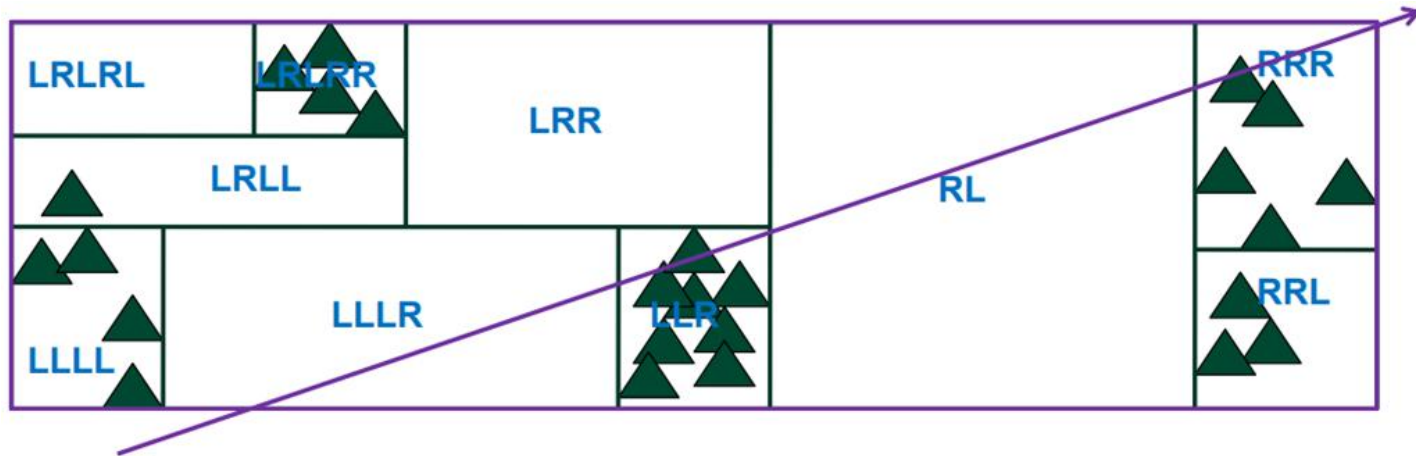


Стек:

Текущий узел:

kd-деревья

- Алгоритм траверса

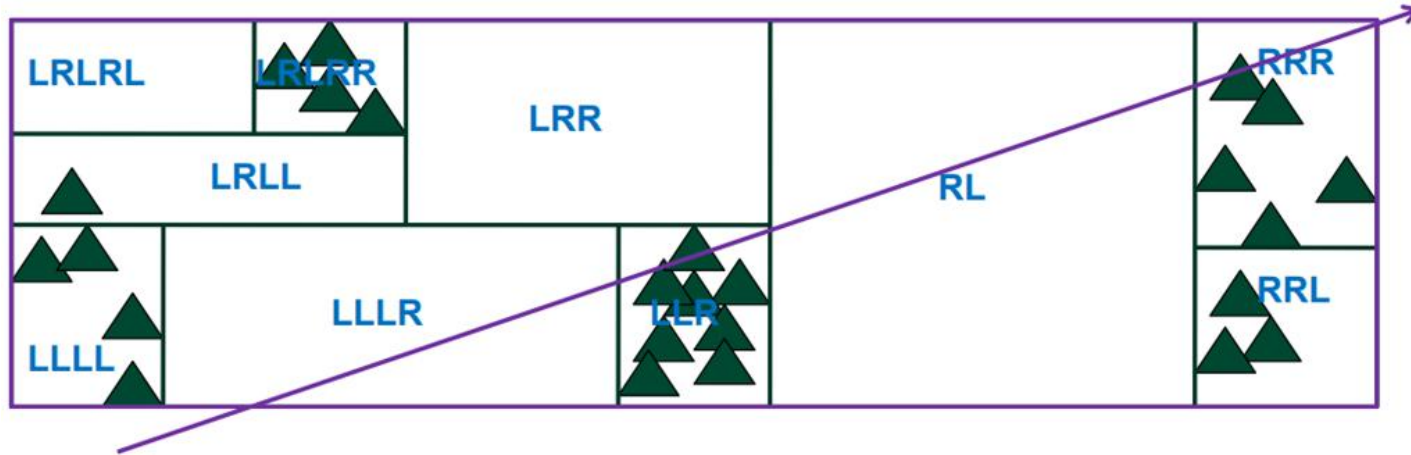


Стек: **R**

Текущий узел: **L**

kd-деревья

- Алгоритм траверса

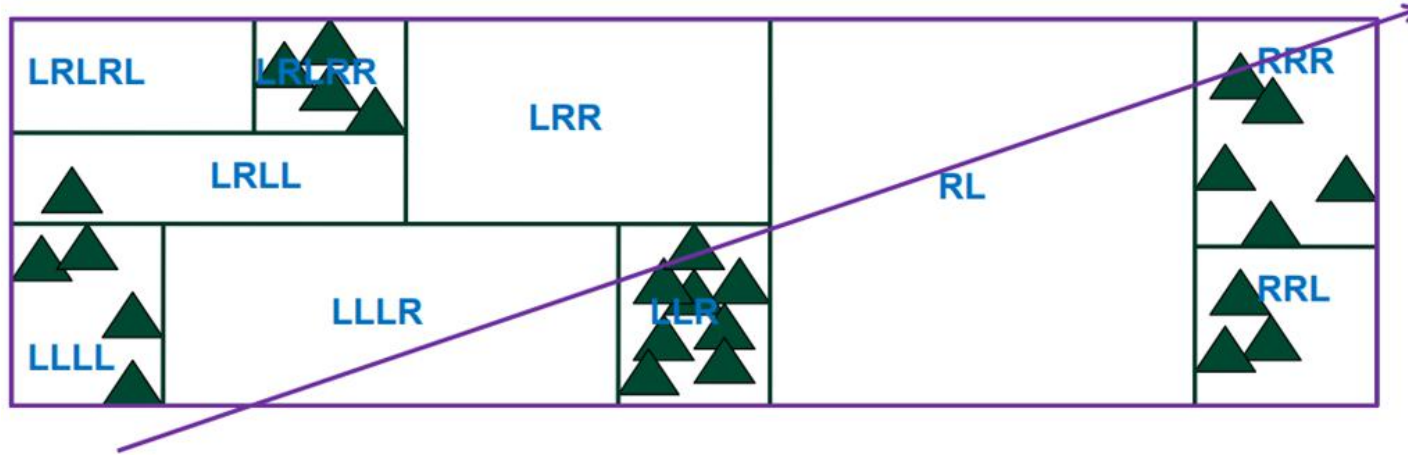


Стек: **R**

Текущий узел: **LL**

kd-деревья

- Алгоритм траверса

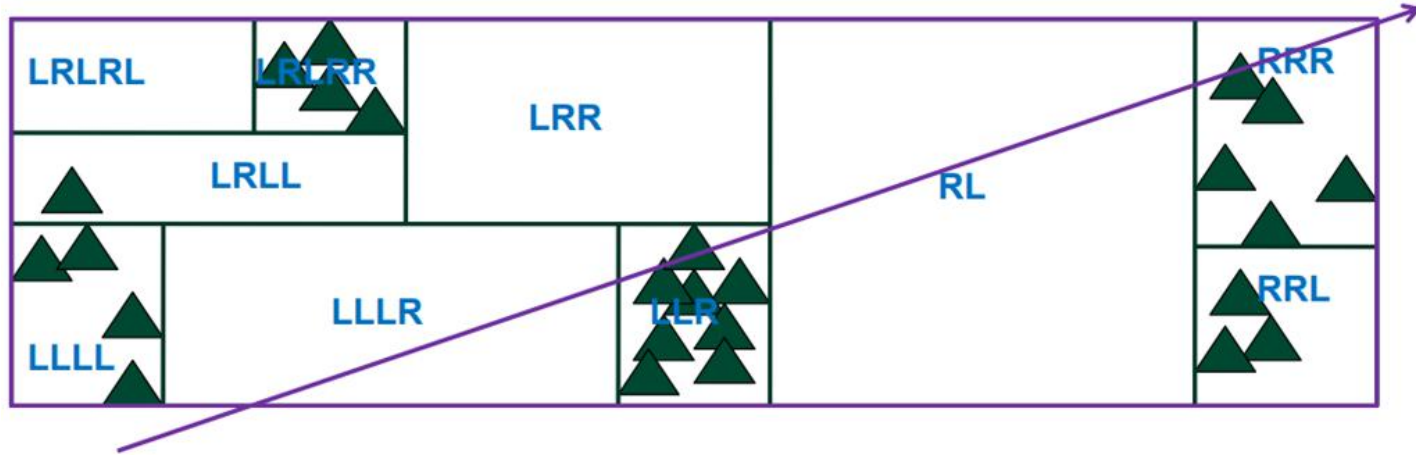


Стек: **LLR, R**

Текущий узел: **LLL**

kd-деревья

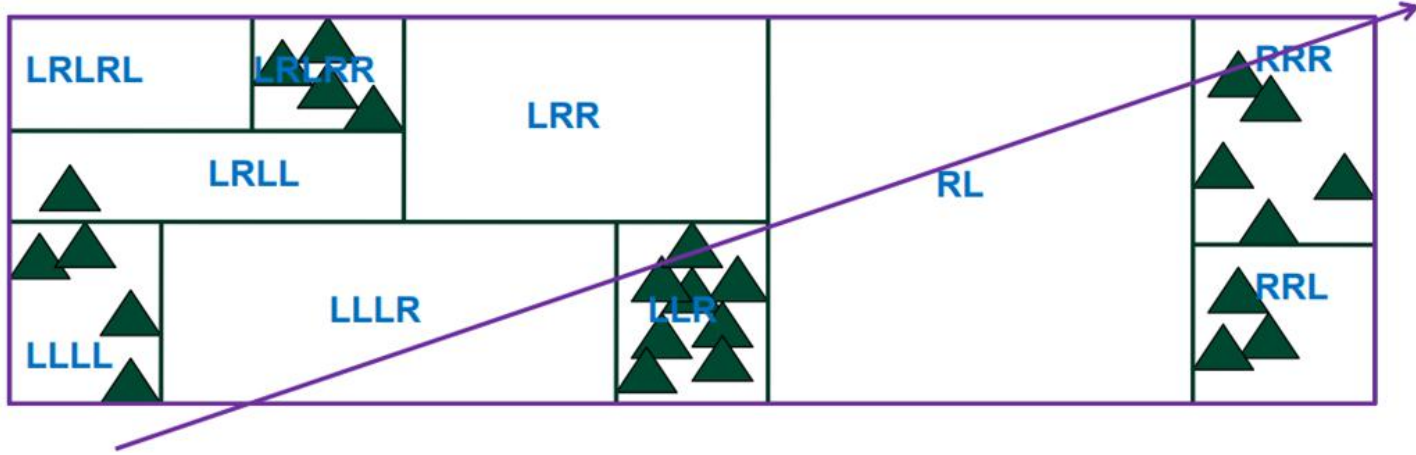
- Алгоритм траверса



Стек: LLR, R
Текущий узел: LLLR

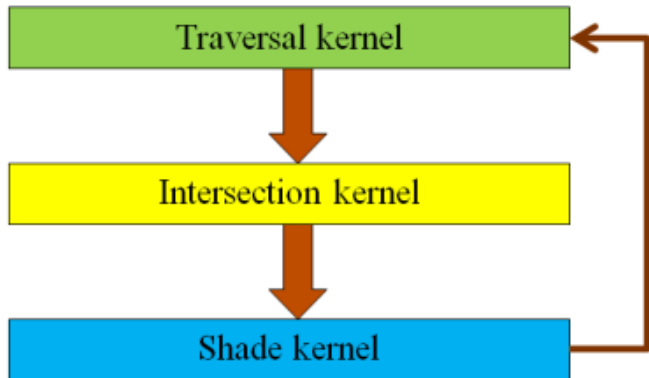
kd-деревья

- Алгоритм траверса



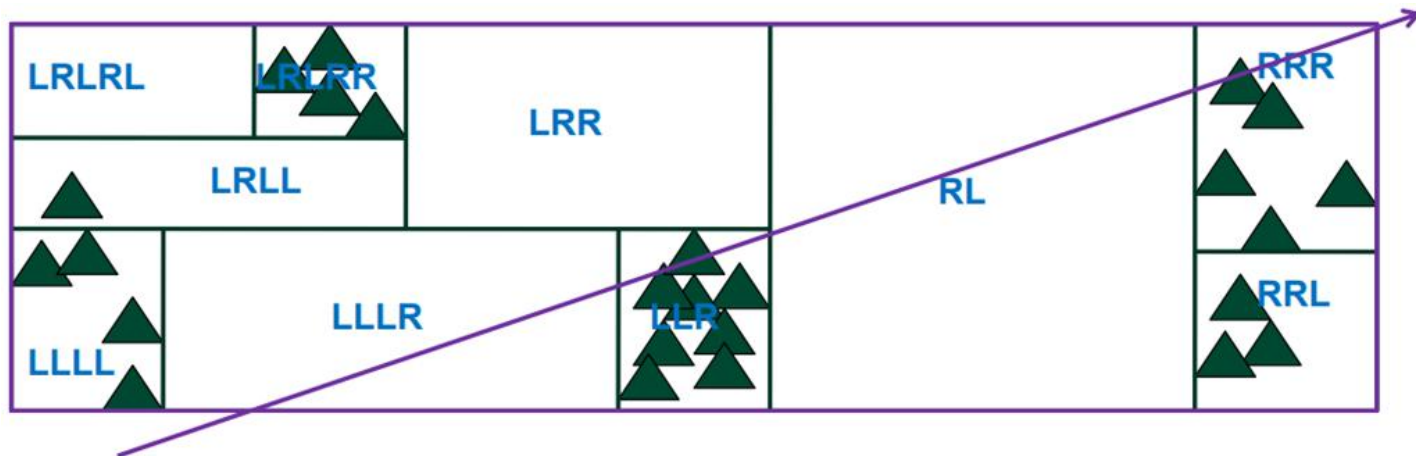

 Стек: **R**
 Текущий узел: **LLR**

Можно было остановиться!



kd-деревья

- Алгоритм траверса

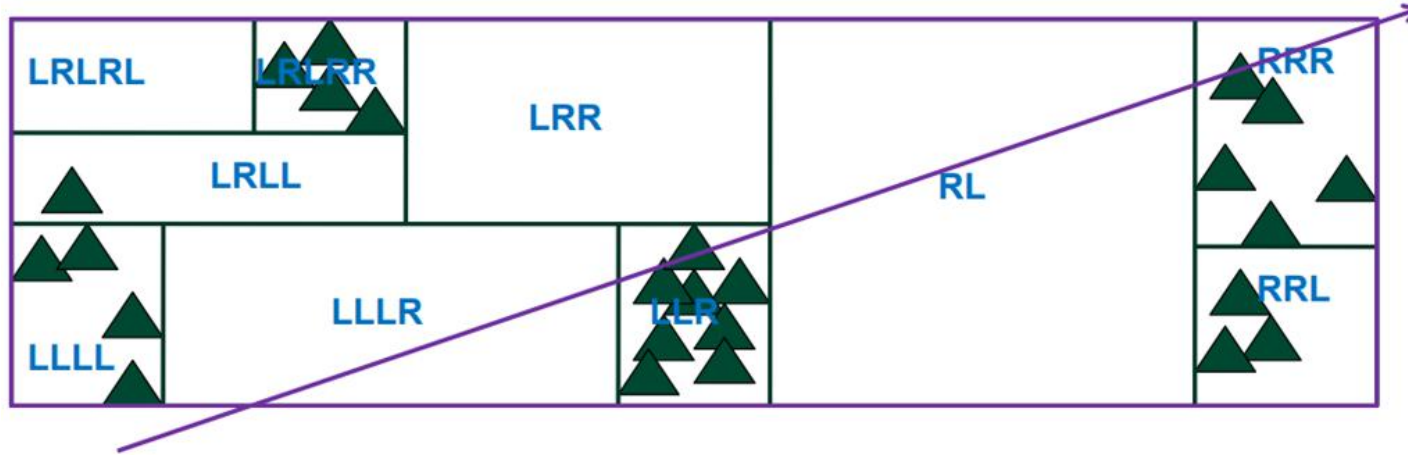


Стек:

Текущий узел: **R**

kd-деревья

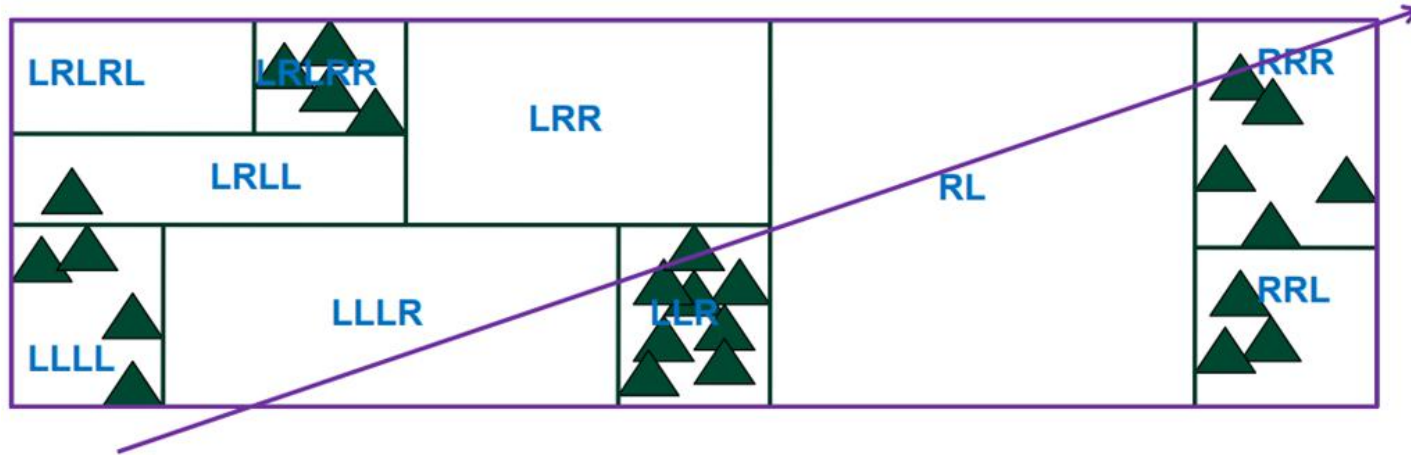
- Алгоритм траверса



Стек: **RR**
Текущий узел: **RL**

kd-деревья

- Алгоритм траверса

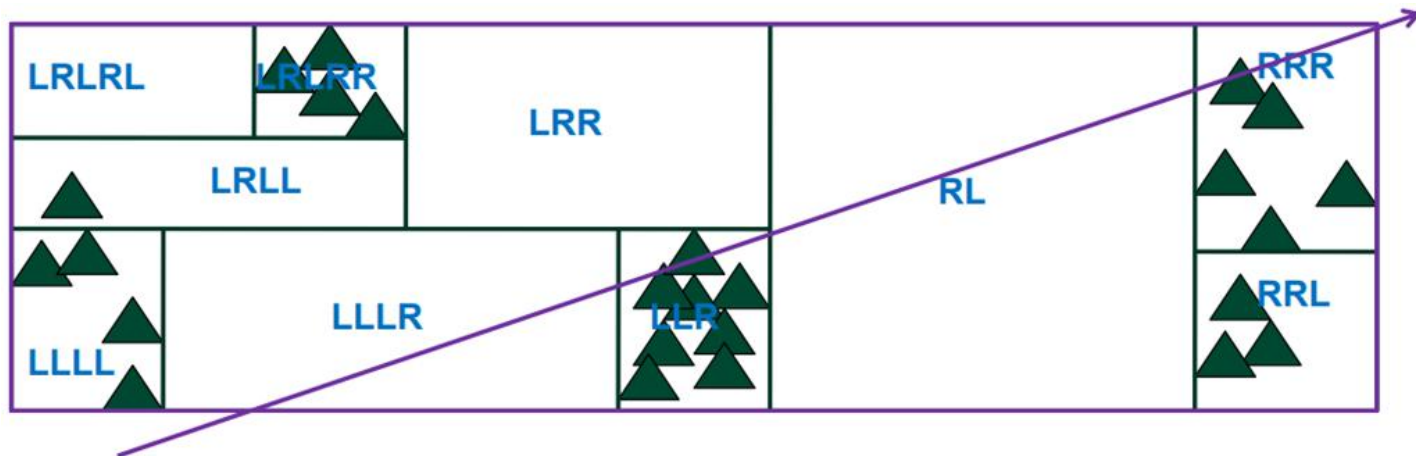


Стек:

Текущий узел: RR

kd-деревья

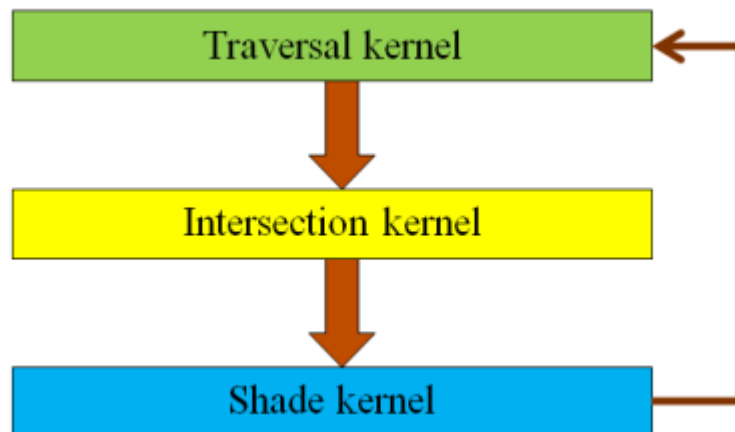
- Алгоритм траверса



Стек:

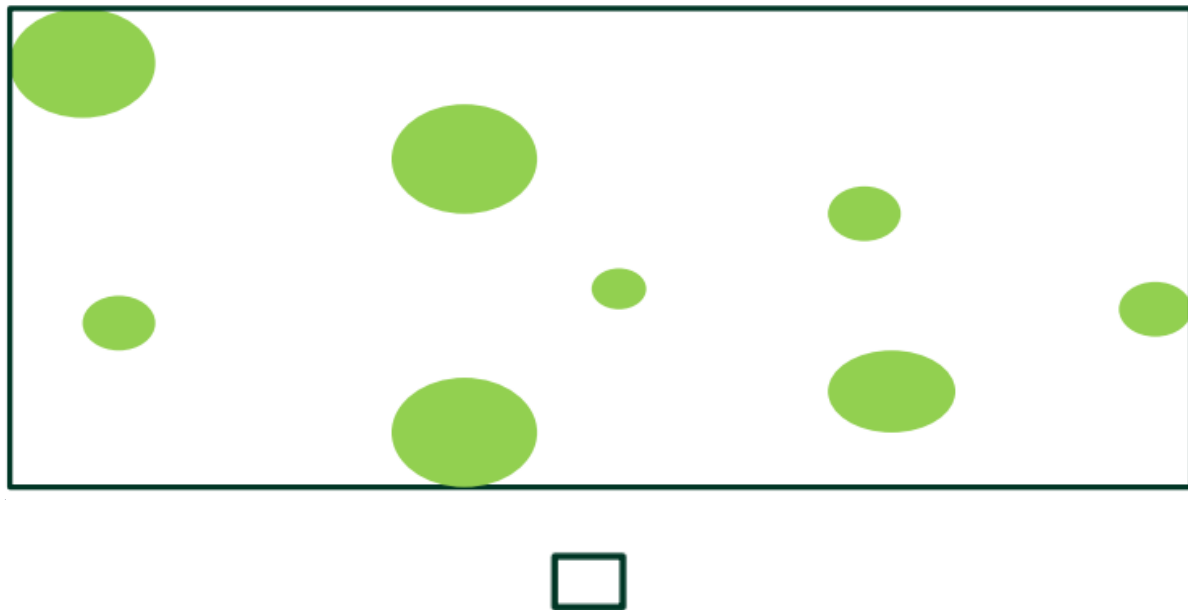
Текущий узел: **RRR**

Конец, результат: **LLR, RRR**



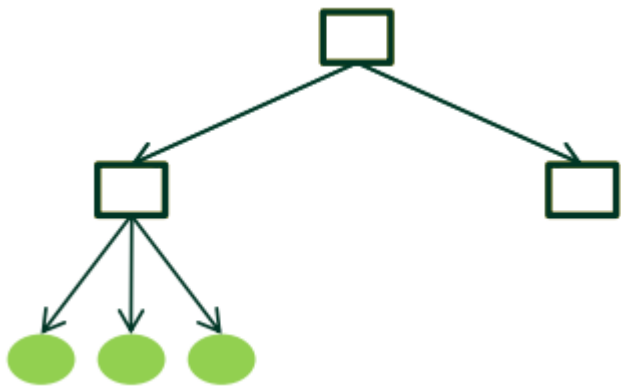
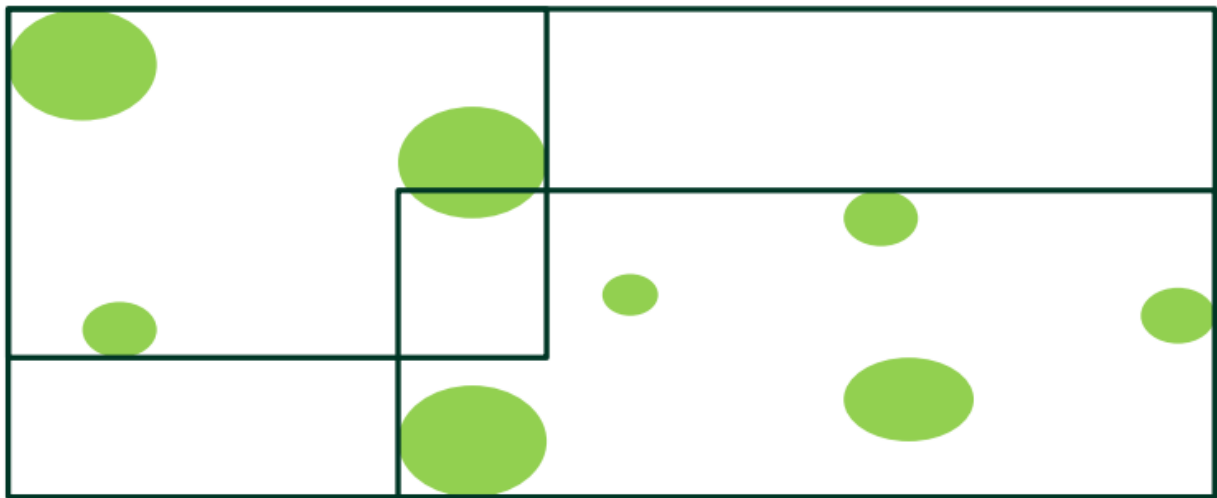
BVH деревья

- Bounding Volume Hierarchy (Иерархия ограничивающих объемов)



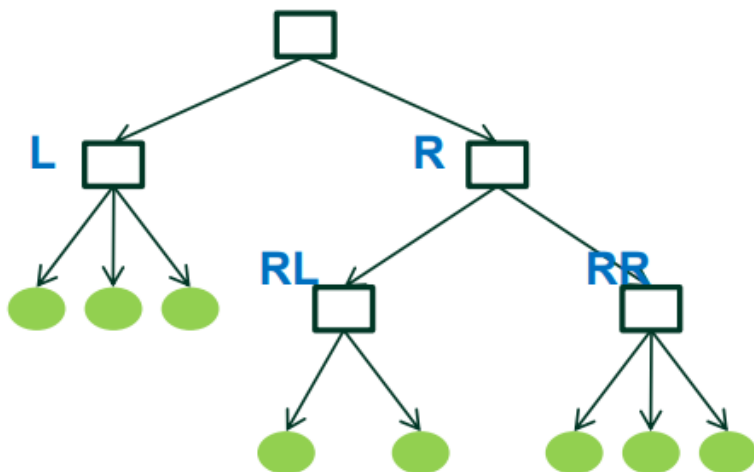
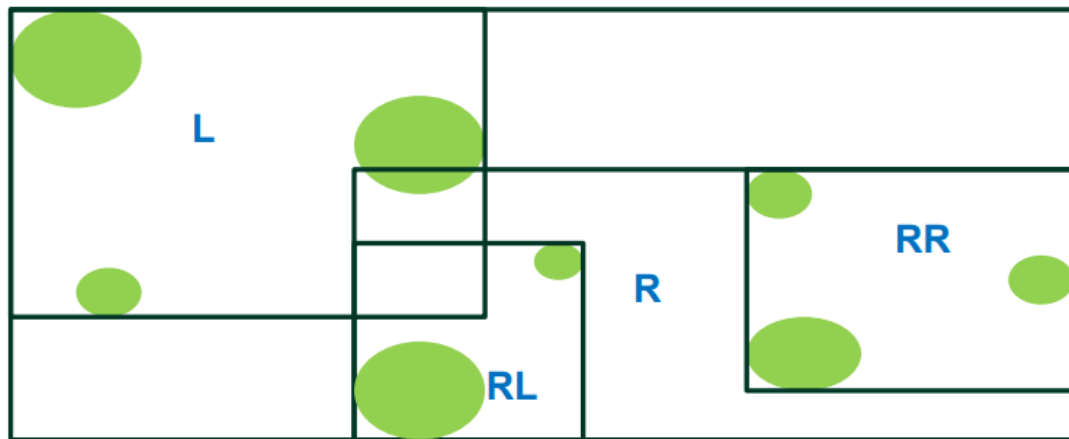
BVH деревья

- Bounding Volume Hierarchy (Иерархия ограничивающих объемов)



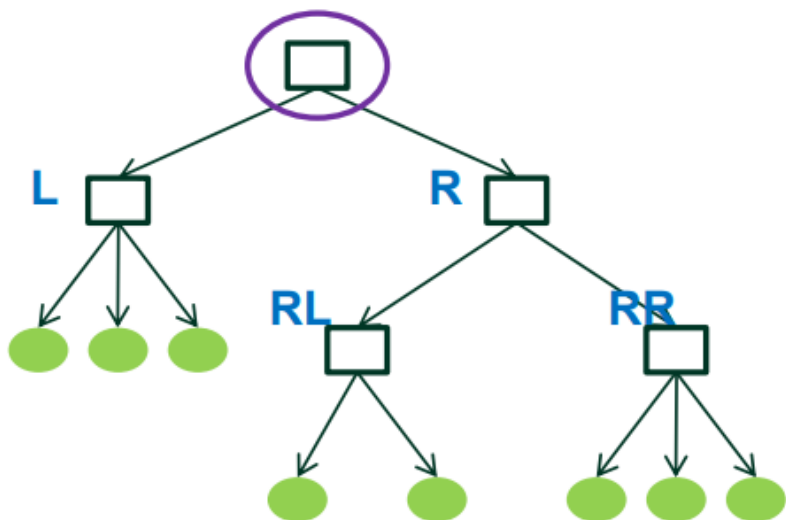
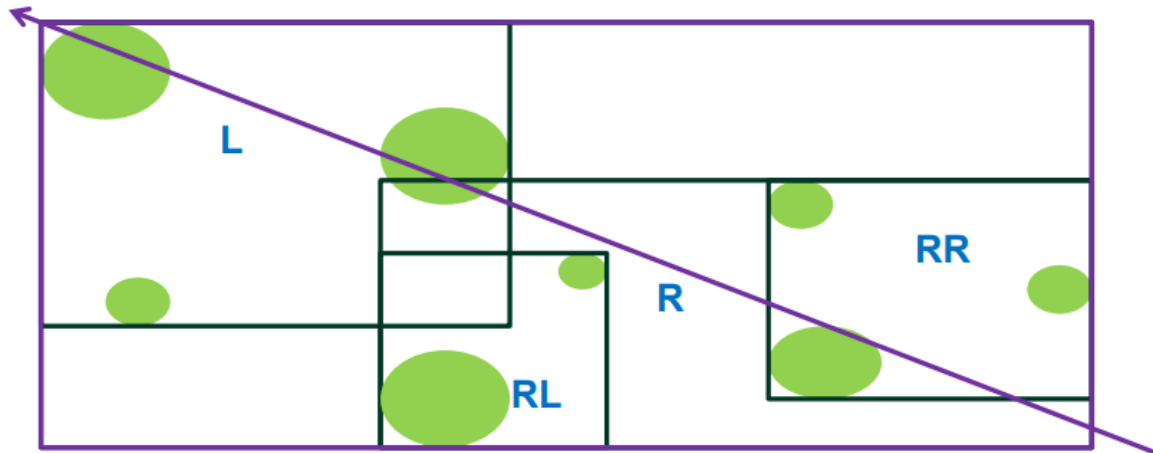
BVH деревья

- Bounding Volume Hierarchy (Иерархия ограничивающих объемов)



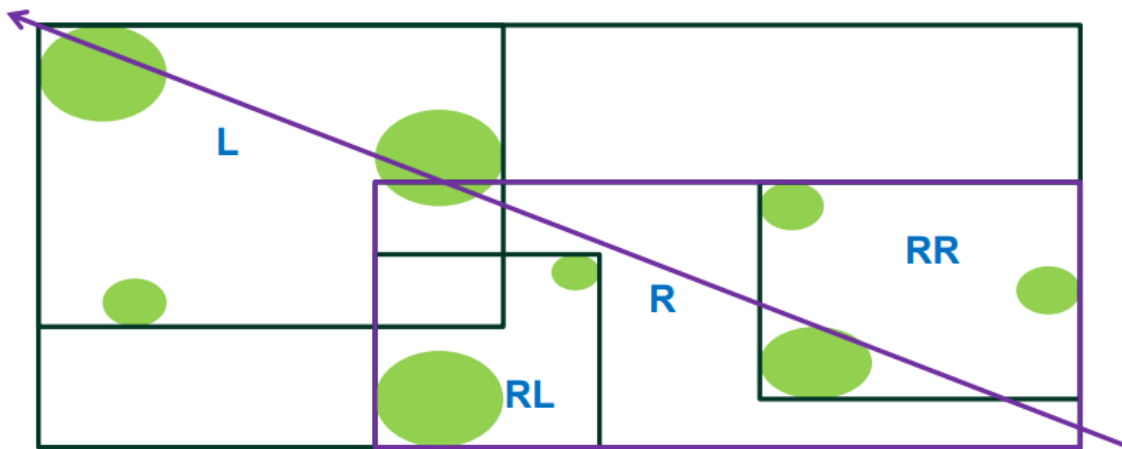
ВН деревья

- Траверс на CPU

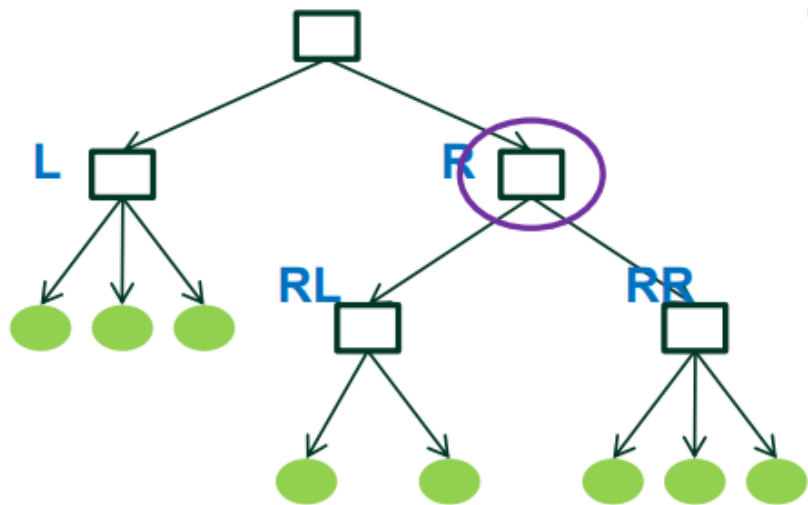


BVH деревья

- Траверс на CPU

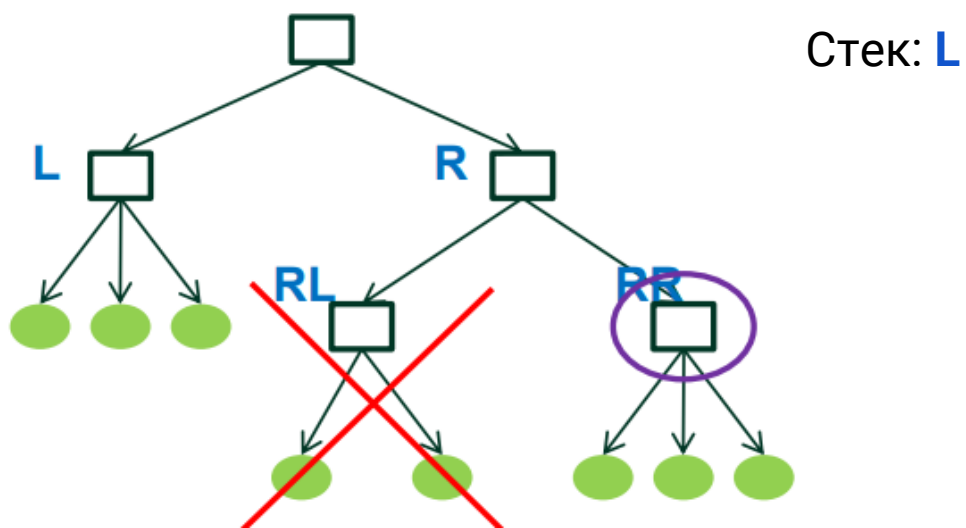
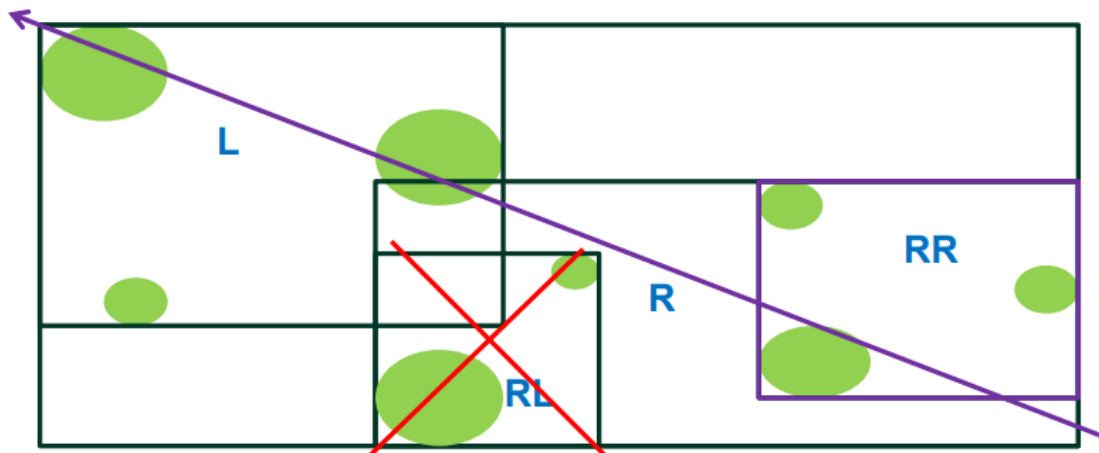


Стек: **L**



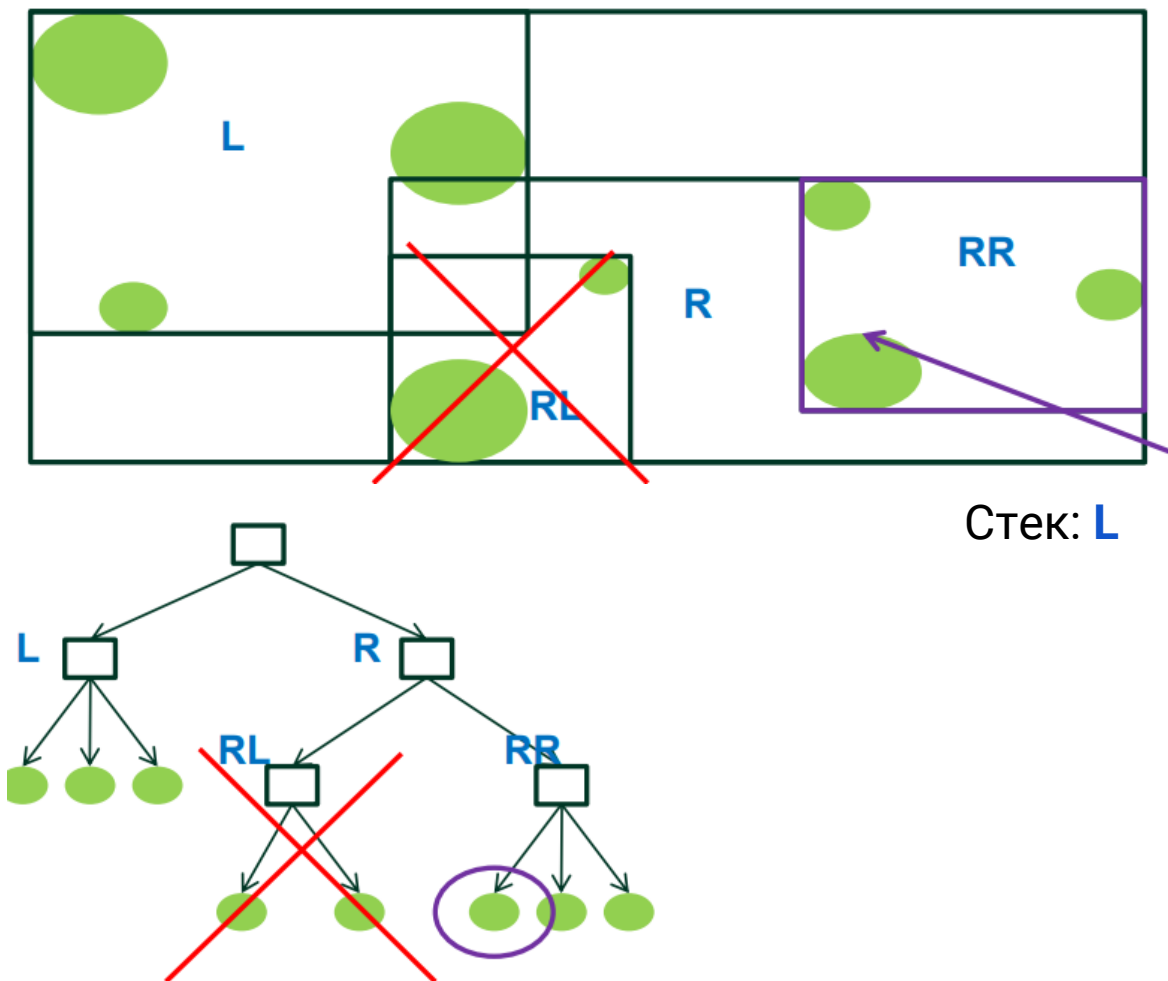
ВН деревья

- Траверс на CPU



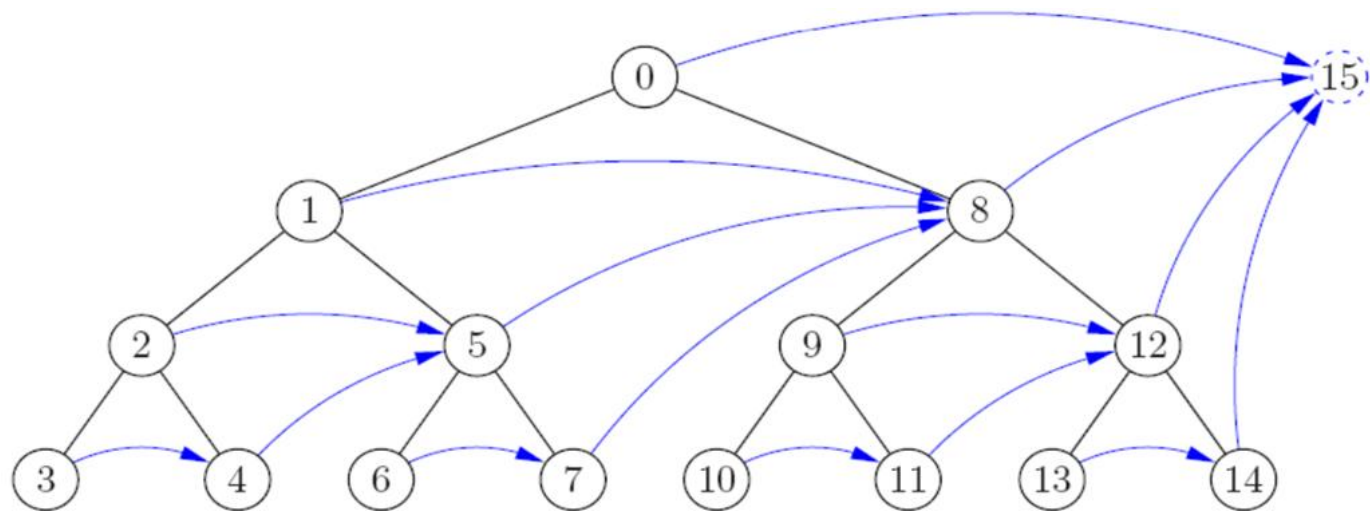
ВН деревья

- Траверс на CPU



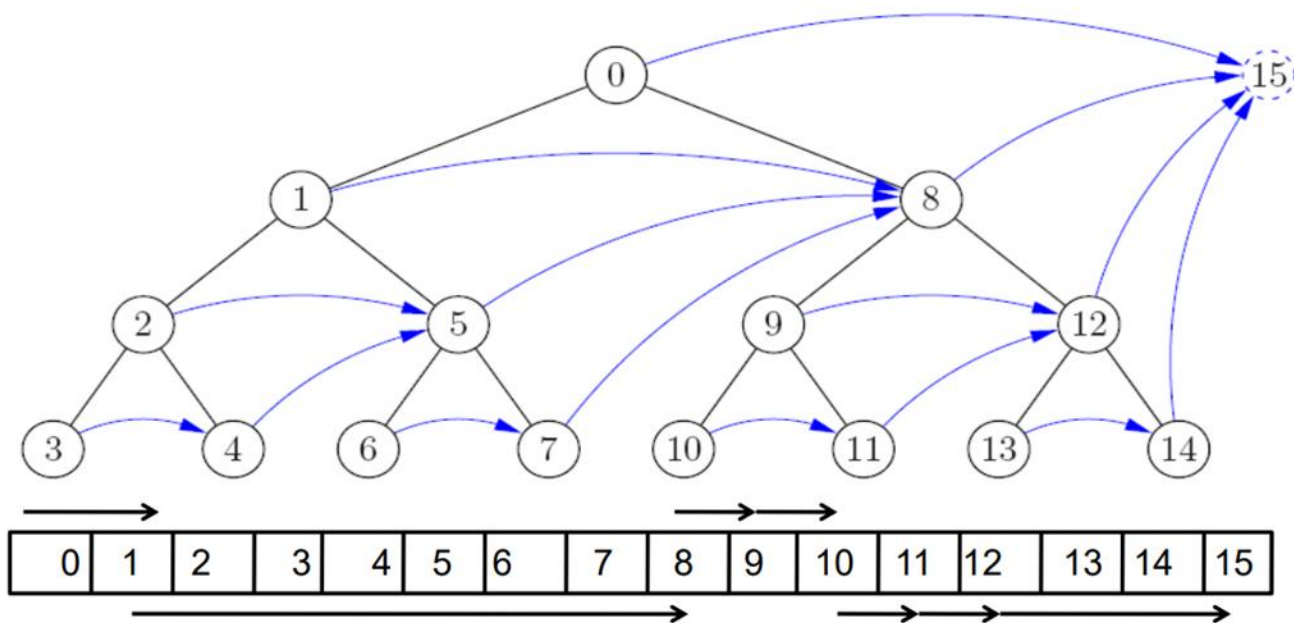
BVH деревья

- Траверс на CPU



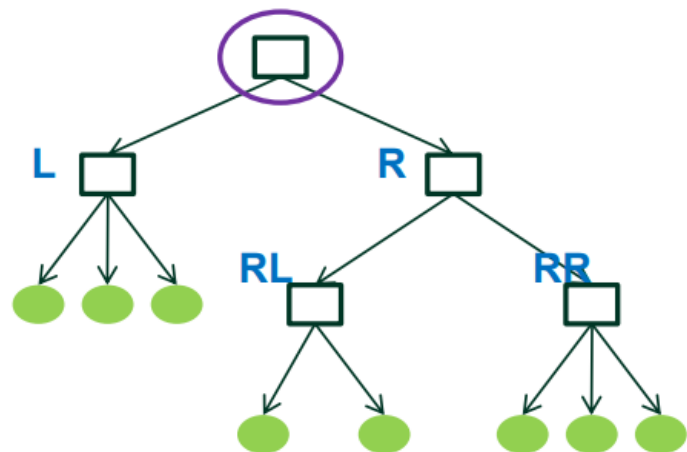
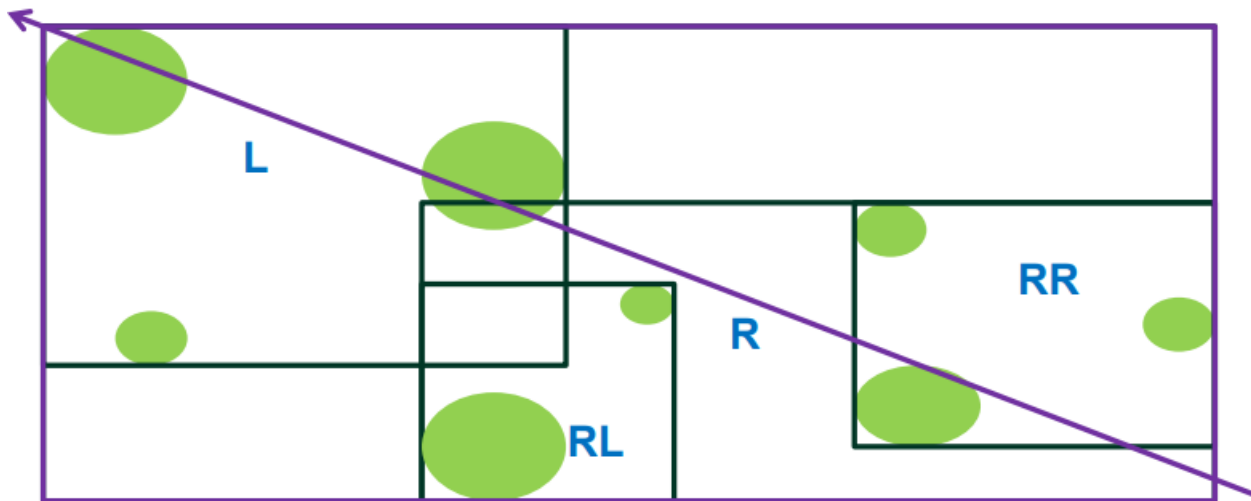
BVH деревья

- Бесстековый траверс на GPU



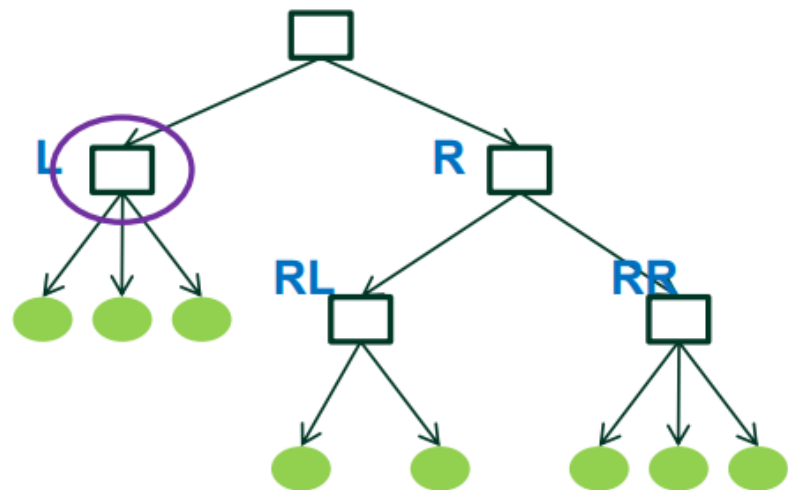
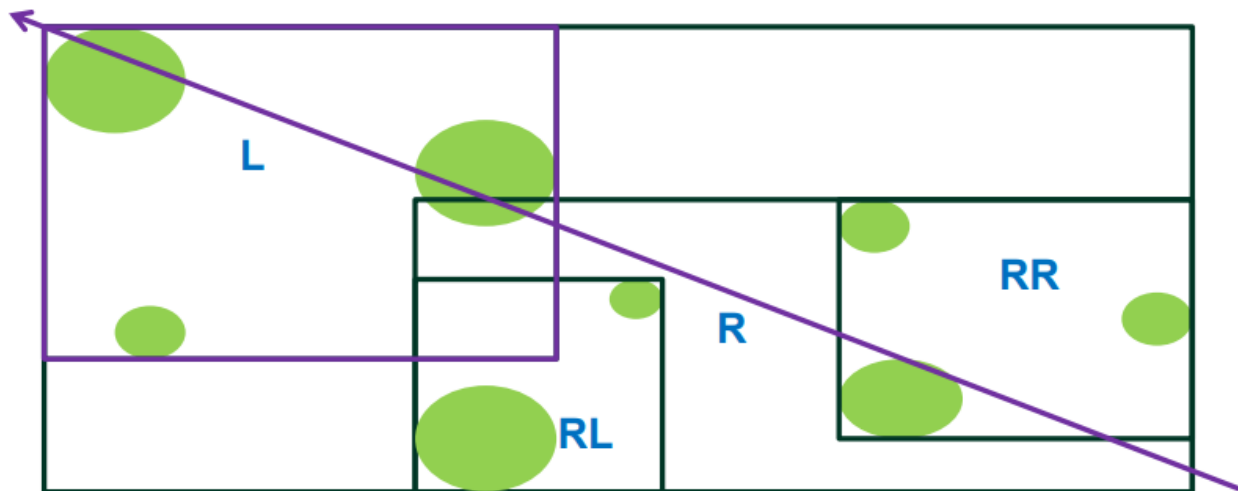
BVH деревья

- Траверс на GPU



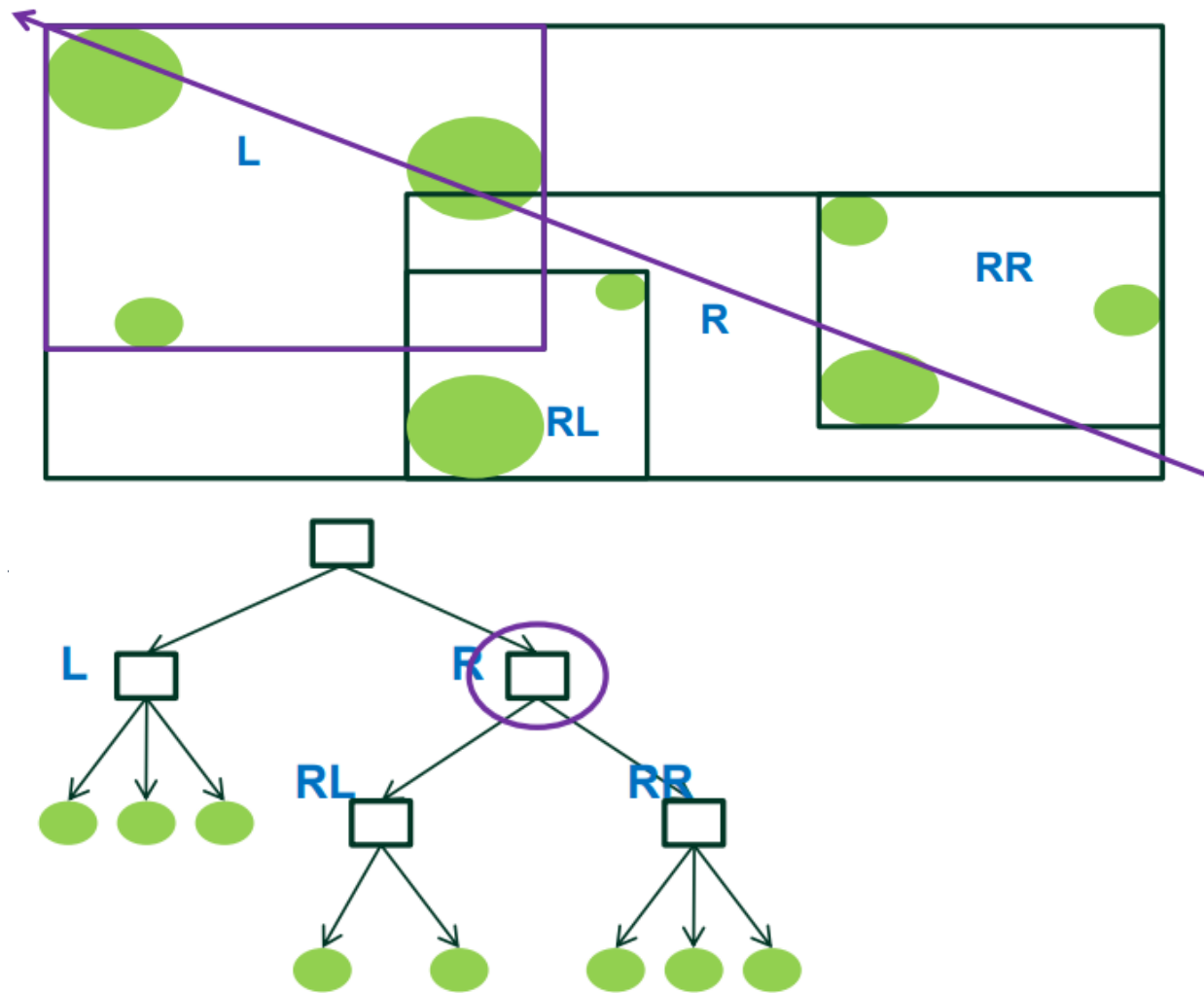
BVH деревья

- Траверс на GPU



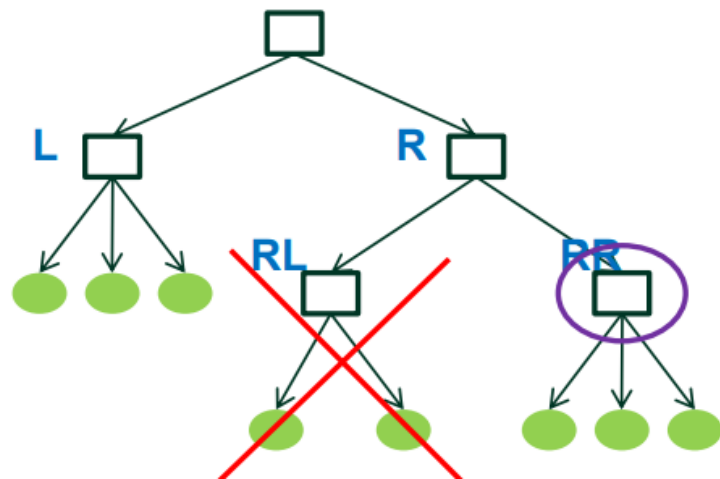
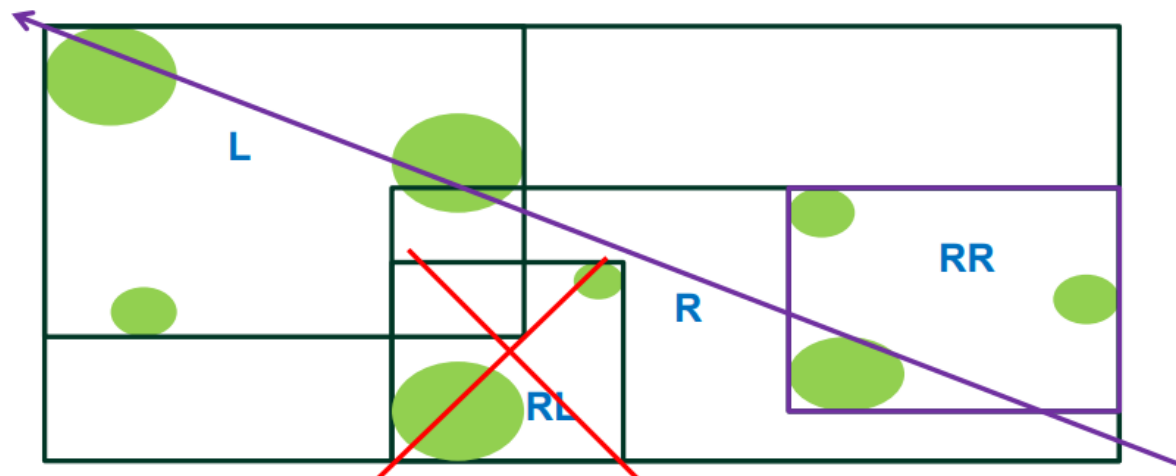
BVH деревья

- Траверс на GPU



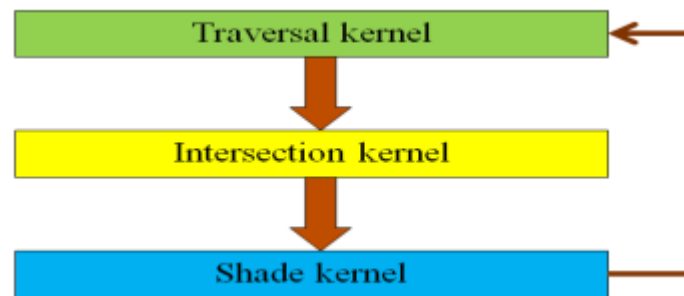
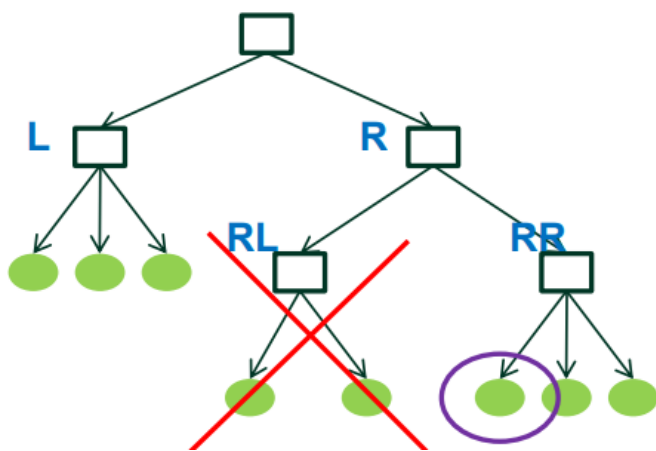
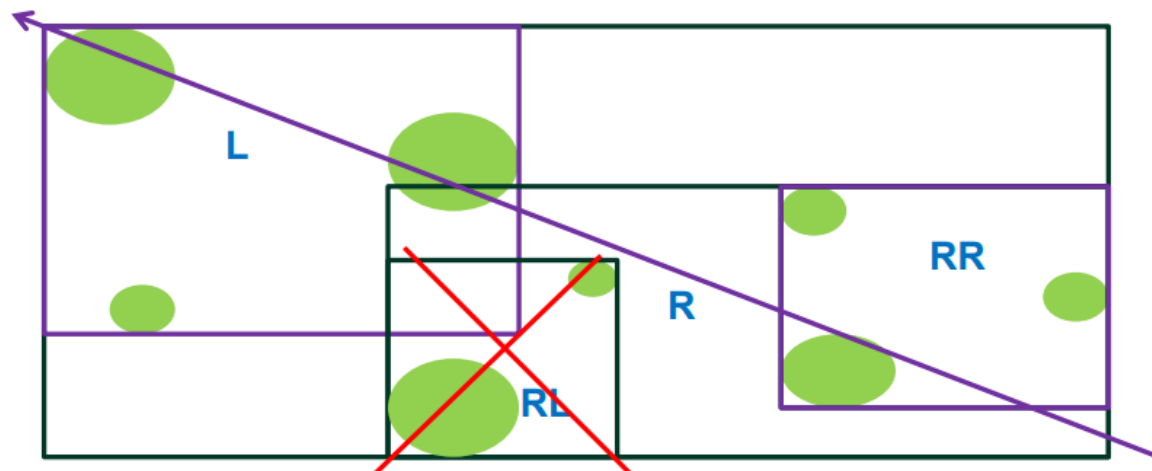
BVH деревья

- Траверс на GPU



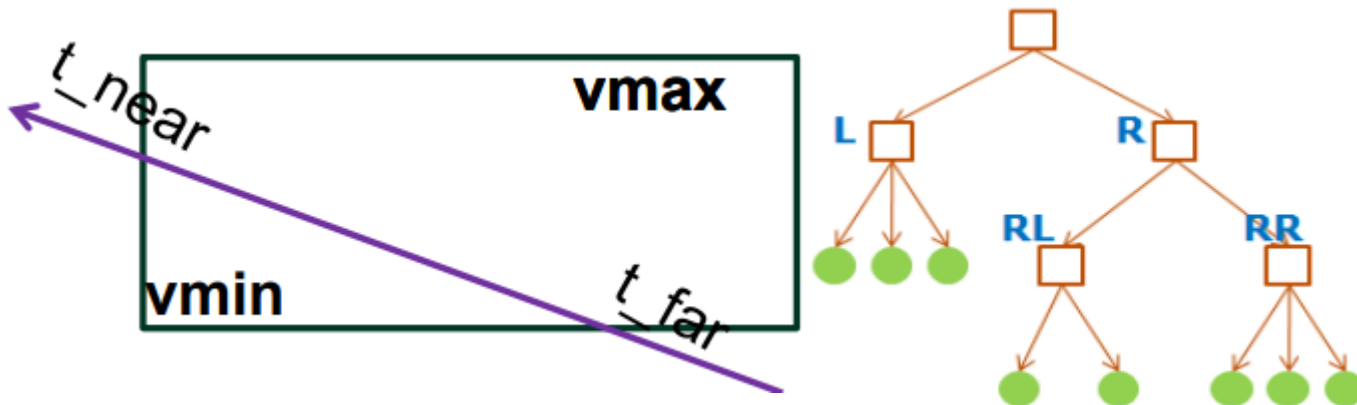
BVH деревья

- Траверс на GPU



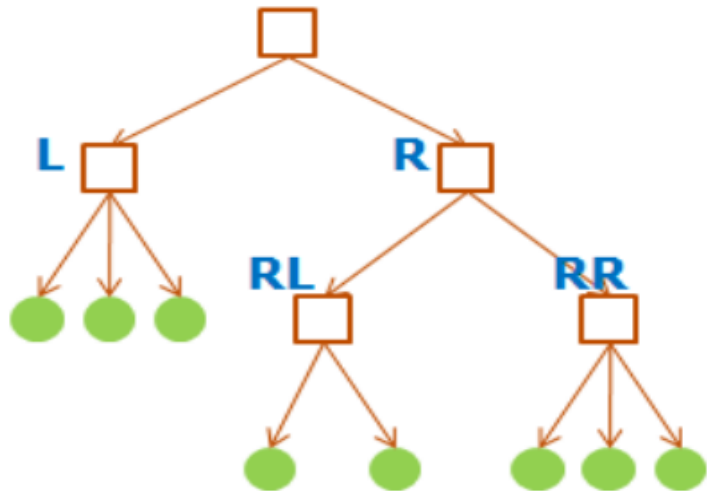
BVH деревья

- Как делать на CUDA?
 - Луч – 6 регистров
 - Бокс – 6 регистров
 - t_near , t_far , - 2
 - $nodeOffset$, $leftOffset$, tid – 3
- Пересечение луча с боксом
 - Минимум по всем 6 плоскостям
 - $(vmin[0] + rInv.pos[0]) * rInv.dir[0];$



BVH деревья

- Как делать на CUDA?
 - Стек на локальной памяти
 - Локальная память это не так медленно, как может показаться
- Бесстековый алгоритм
 - Перебираем массив всегда строго слева направо



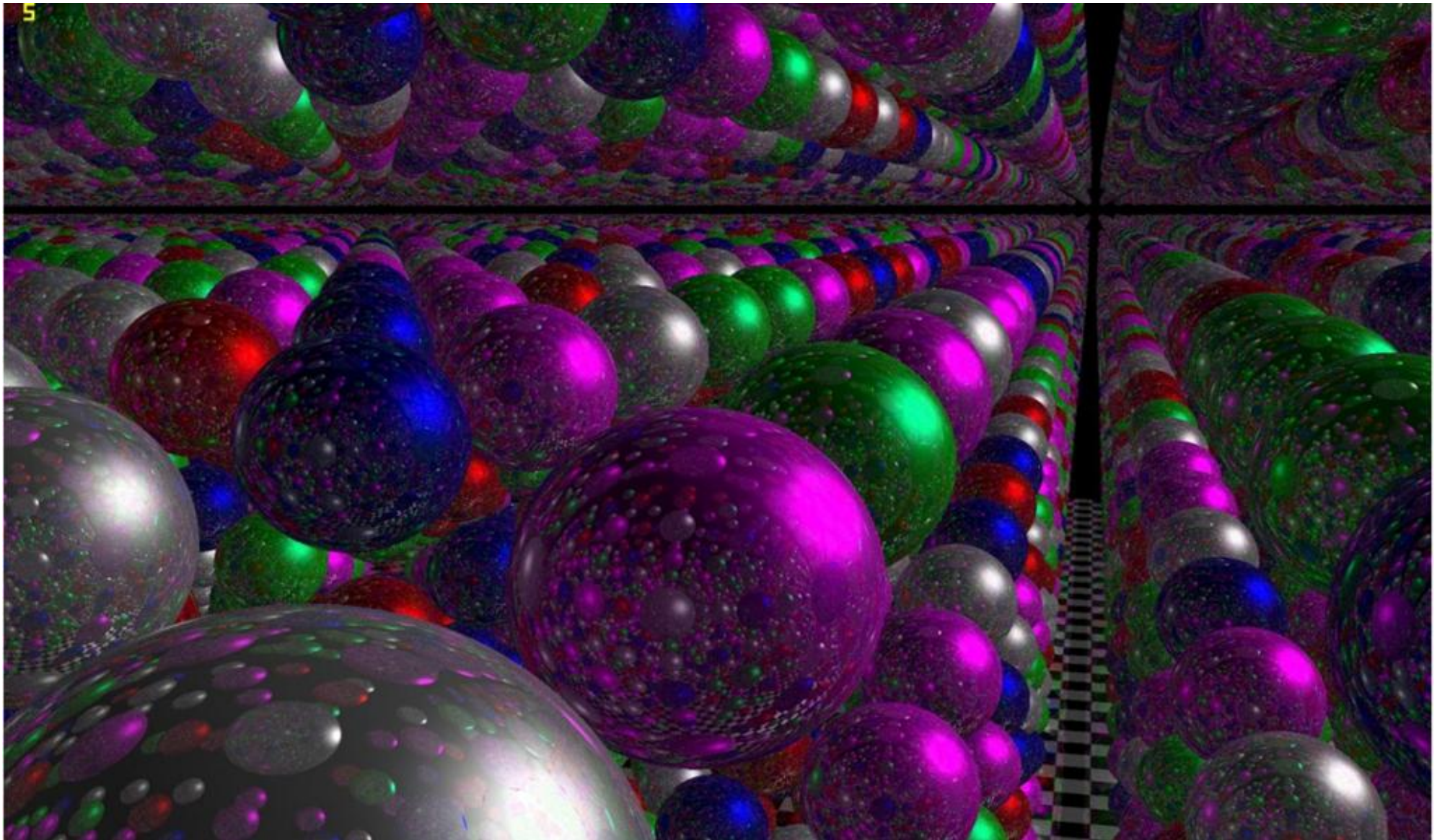
Улучшаем производительность

- Persistent threads

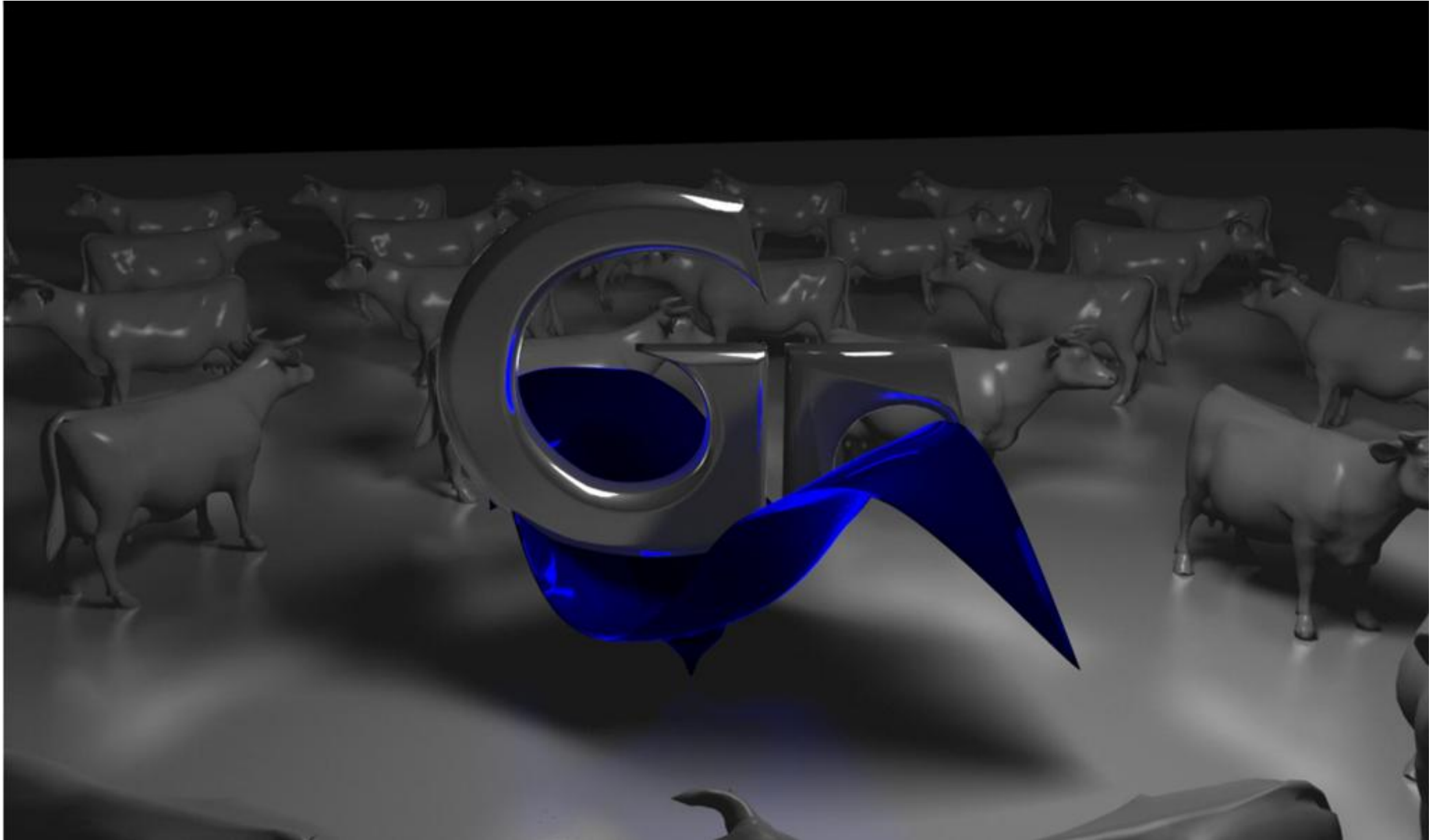
```
__global__ void my_kernel(){  
    for (int warp = 0; int warp < 16; warp++){  
        int tid = blockDim.x * blockIdx.x + threadIdx.x +  
            warp * step;  
        ... // тут код вашего кёрнела  
    }  
}
```



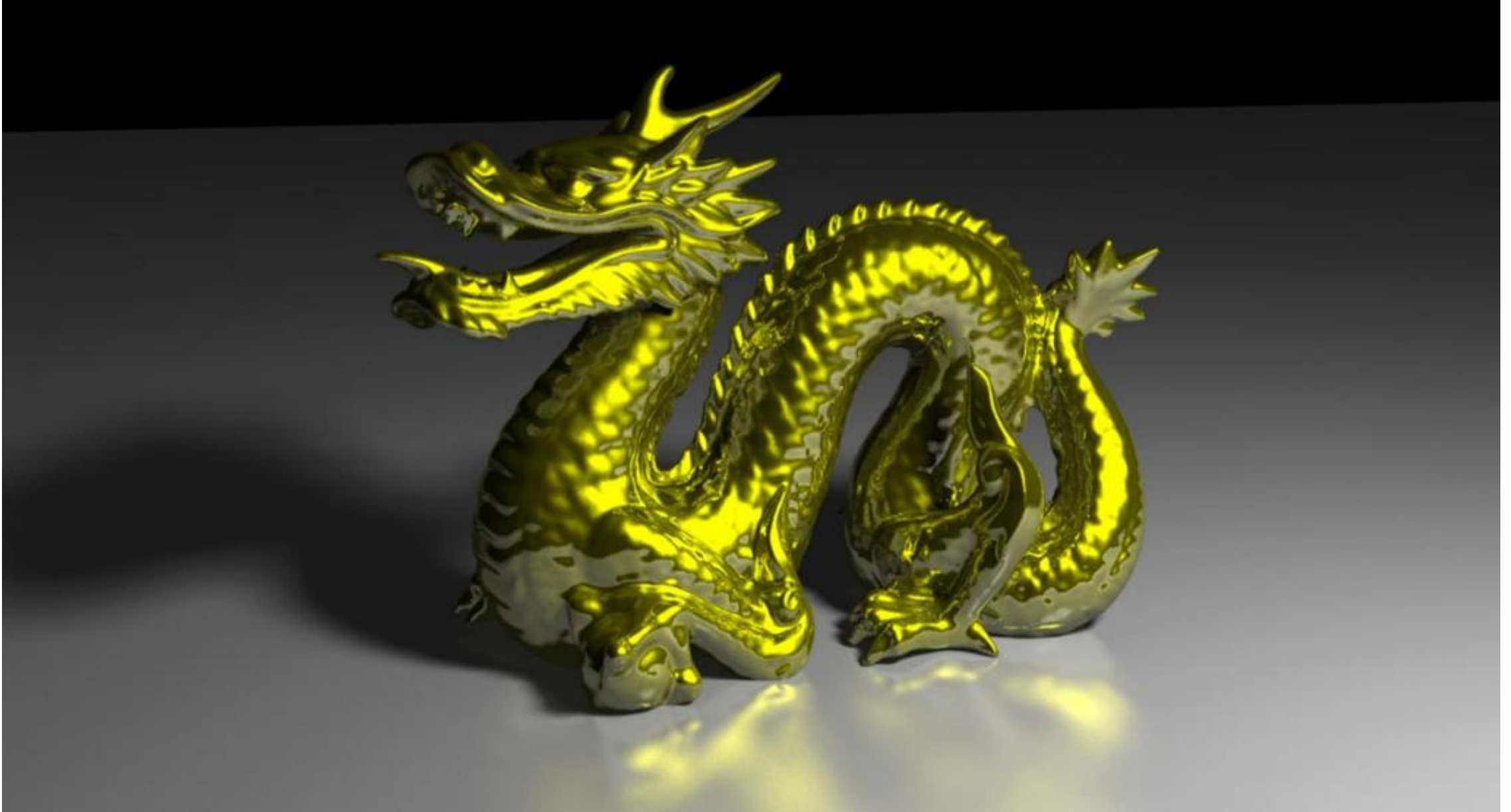
360К сфер, 3 отражения,
1920x1200



Path tracing



Path tracing



RayTracing